Cryptography

Lecture 8: Practical Symmetric-Key Primitives

Arkady Yerukhimovich

October 26, 2026

Exam 1 Grades

Exam 1 Stats:

- Max – 91
- Mean – 62.76
- Median – 61

Optional Make-up:

- You have the option to retake the exam as a homework assignment.
- For any problem, on which you improve, you will receive 1/2 of the difference
- You cannot lose points
- The make-up is due by 12:45pm next Monday

Where Are We Now?

We have covered:

- Cryptographic building blocks: PRGs, PRFs, and collision-resistant hash functions.
- Symmetric-key crypto:
  - Encryption: CPA security, modes of operation, and authenticated encryption.
  - MACs for message authentication.
  - How to build these from PRFs and collision-resistant hash functions.

Where Are We Now?

We have covered:

- Cryptographic building blocks: PRGs, PRFs, and collision-resistant hash functions.
- Symmetric-key crypto:
  - Encryption: CPA security, modes of operation, and authenticated encryption.
  - MACs for message authentication.
  - How to build these from PRFs and collision-resistant hash functions.

What we are missing:

- How to instantiate these building blocks efficiently – TODAY!
- How to go beyond symmetric-key crypto – Starting next week

And Now For Something Completely Different

- This week, we will step away from asymptotic notions of security

And Now For Something Completely Different

- This week, we will step away from asymptotic notions of security
- Focus on constructing *practically efficient* cryptographic objects

And Now For Something Completely Different

- This week, we will step away from asymptotic notions of security
- Focus on constructing *practically efficient* cryptographic objects
- Constructions will be largely combinatorial, and proofs informal

Outline

1. Constructing practical block ciphers
2. Substitution–permutation networks and AES
3. A bridge to practical hash functions
4. Sponge functions
5. Keccak, SHA-3, and SHAKE

What Is a Block Cipher?

A family of efficiently invertible permutations

$F : \{0,1\}^n \times \{0,1\}^\ell \to \{0,1\}^\ell$; write $F_k(x)$.

- n: key length; ℓ: block length.
- For each fixed key, F_k is a permutation of the blocks.
- Knowing k allows both evaluation and inversion.

What Is a Block Cipher?

A family of efficiently invertible permutations

$F : \{0,1\}^n \times \{0,1\}^\ell \to \{0,1\}^\ell$; write $F_k(x)$.

- n: key length; ℓ: block length.
- For each fixed key, F_k is a permutation of the blocks.
- Knowing k allows both evaluation and inversion.

AES: $\ell = 128$; $n \in \{128, 192, 256\}$.

What Is a Block Cipher?

A family of efficiently invertible permutations

$F : \{0,1\}^n \times \{0,1\}^\ell \to \{0,1\}^\ell$; write $F_k(x)$.

- n: key length; ℓ: block length.
- For each fixed key, F_k is a permutation of the blocks.
- Knowing k allows both evaluation and inversion.

AES: $\ell = 128$; $n \in \{128, 192, 256\}$.

Remember

A block cipher is a building block. Encrypting messages still requires a mode of operation, with its own security requirements.

What Security Are We Aiming For?

- PRP security: oracle access should look like access to a random permutation.

What Security Are We Aiming For?

- PRP security: oracle access should look like access to a random permutation.
- Strong PRP security also allows queries to the inverse oracle.

What Security Are We Aiming For?

- PRP security: oracle access should look like access to a random permutation.
- Strong PRP security also allows queries to the inverse oracle.
- Exhaustive key search takes about 2^n work; we aim to prevent substantially cheaper key recovery.

What Security Are We Aiming For?

- PRP security: oracle access should look like access to a random permutation.
- Strong PRP security also allows queries to the inverse oracle.
- Exhaustive key search takes about 2^n work; we aim to prevent substantially cheaper key recovery.
- This is not a blanket 2^n security guarantee: the block length and the surrounding mode also limit how much data we can safely process.

What Security Are We Aiming For?

- PRP security: oracle access should look like access to a random permutation.
- Strong PRP security also allows queries to the inverse oracle.
- Exhaustive key search takes about 2^n work; we aim to prevent substantially cheaper key recovery.
- This is not a blanket 2^n security guarantee: the block length and the surrounding mode also limit how much data we can safely process.

Evidence, rather than a proof by reduction

Public design, extensive cryptanalysis, and a margin between attacks on reduced-round variants and the full construction.

Building a Block Cipher

The Challenge

There are $(2^\ell)!$ different permutations on ℓ bits

Building a Block Cipher

The Challenge

There are $(2^\ell)!$ different permutations on ℓ bits

- Writing down a permutation takes $\log (2^\ell)! \approx \ell \cdot 2^\ell$ bits
  We use $\ell \geq 128$, far too large to store

Building a Block Cipher

The Challenge

There are $(2^\ell)!$ different permutations on ℓ bits

- Writing down a permutation takes $\log(2^\ell)! \approx \ell \cdot 2^\ell$ bits
  We use $\ell \geq 128$, far too large to store
- Need a short description
  Otherwise, how do you evaluate efficiently?

Building a Block Cipher

The Challenge

There are $(2^\ell)!$ different permutations on ℓ bits

- Writing down a permutation takes $\log(2^\ell)! \approx \ell \cdot 2^\ell$ bits
  We use $\ell \geq 128$, far too large to store
- Need a short description
  Otherwise, how do you evaluate efficiently?

Solution Idea: Instead of one permutation on ℓ bits, compose many small (e.g., 8-bit) permutations

$$F_k(x) = f_1(x_1)||f_2(x_2)||\cdots||f_{16}(x_{16})$$

Building a Block Cipher

The Challenge

There are $(2^\ell)!$ different permutations on ℓ bits

- Writing down a permutation takes $\log(2^\ell)! \approx \ell \cdot 2^\ell$ bits
  We use $\ell \geq 128$, far too large to store
- Need a short description
  Otherwise, how do you evaluate efficiently?

Solution Idea: Instead of one permutation on ℓ bits, compose many small (e.g., 8-bit) permutations

$$F_k(x) = f_1(x_1)||f_2(x_2)||\cdots||f_{16}(x_{16})$$

- Pro: $|F| = 16 \cdot \log((2^8)!) \approx 16 \cdot 1600$ bits $\approx 3KB << 128 \cdot 2^{128}$

Building a Block Cipher

The Challenge

There are $(2^\ell)!$ different permutations on ℓ bits

- Writing down a permutation takes $\log(2^\ell)! \approx \ell \cdot 2^\ell$ bits
  We use $\ell \geq 128$, far too large to store
- Need a short description
  Otherwise, how do you evaluate efficiently?

Solution Idea: Instead of one permutation on ℓ bits, compose many small (e.g., 8-bit) permutations

$$F_k(x) = f_1(x_1)||f_2(x_2)||\cdots||f_{16}(x_{16})$$

- Pro: $|F| = 16 \cdot \log((2^8)!) \approx 16 \cdot 1600$ bits $\approx 3KB << 128 \cdot 2^{128}$
- Con: If x only changes in one bit, only one block is affected

Avalanche Effect

A critical goal in design of block-ciphers is to achieve something called the *avalanche effect*.

Avalanche Effect

Small change in the input must "affect" every bit of the output.

Avalanche Effect

A critical goal in design of block-ciphers is to achieve something called the *avalanche effect*.

Avalanche Effect

Small change in the input must "affect" every bit of the output.

What this means:

Avalanche Effect

A critical goal in design of block-ciphers is to achieve something called the *avalanche effect*.

Avalanche Effect

Small change in the input must "affect" every bit of the output.

What this means:

- This does not require that all bits of output change, only that the bits of the output are not statistically independent of the change in input.

Avalanche Effect

A critical goal in design of block-ciphers is to achieve something called the *avalanche effect*.

Avalanche Effect

Small change in the input must "affect" every bit of the output.

What this means:

- This does not require that all bits of output change, only that the bits of the output are not statistically independent of the change in input.
- Even a 1-bit change to input must have statistical impact on all bits.

Avalanche Effect

A critical goal in design of block-ciphers is to achieve something called the *avalanche effect*.

Avalanche Effect

Small change in the input must "affect" every bit of the output.

What this means:

- This does not require that all bits of output change, only that the bits of the output are not statistically independent of the change in input.
- Even a 1-bit change to input must have statistical impact on all bits.
- Ideally, 1-bit change in input should change about half of the output bits.

Confusion-Diffusion Paradigm

Many block-ciphers follow the following general approach:

Confusion-Diffusion Paradigm

Many block-ciphers follow the following general approach:

- Confusion: $F_k(x) = f_1(x_1)||f_2(x_2)||\cdots||f_{16}(x_{16})$
  - $|x_i| = 8$ bits
  - f_i can be a random permutation on 8 bits
  - Goal: Introduce non-linear local changes

Confusion-Diffusion Paradigm

Many block-ciphers follow the following general approach:

- Confusion: $F_k(x) = f_1(x_1)||f_2(x_2)||\cdots||f_{16}(x_{16})$
  - $|x_i| = 8$ bits
  - f_i can be a random permutation on 8 bits
  - Goal: Introduce non-linear local changes
- Diffusion: Shuffle the bits using a *mixing permutation*
  - This just moves bits around
  - Goal: Spread confusion across all 128 bits

Confusion-Diffusion Paradigm

Many block-ciphers follow the following general approach:

- Confusion: $F_k(x) = f_1(x_1)||f_2(x_2)||\cdots||f_{16}(x_{16})$
  - $|x_i| = 8$ bits
  - f_i can be a random permutation on 8 bits
  - Goal: Introduce non-linear local changes
- Diffusion: Shuffle the bits using a *mixing permutation*
  - This just moves bits around
  - Goal: Spread confusion across all 128 bits
- Repeat many times

Confusion-Diffusion Paradigm

Many block-ciphers follow the following general approach:

- Confusion: $F_k(x) = f_1(x_1)||f_2(x_2)||\cdots||f_{16}(x_{16})$
  - $|x_i| = 8$ bits
  - f_i can be a random permutation on 8 bits
  - Goal: Introduce non-linear local changes
- Diffusion: Shuffle the bits using a *mixing permutation*
  - This just moves bits around
  - Goal: Spread confusion across all 128 bits
- Repeat many times

Intuition:

Confusion-Diffusion Paradigm

Many block-ciphers follow the following general approach:

- Confusion: $F_k(x) = f_1(x_1)||f_2(x_2)||\cdots||f_{16}(x_{16})$
  - $|x_i| = 8$ bits
  - f_i can be a random permutation on 8 bits
  - Goal: Introduce non-linear local changes
- Diffusion: Shuffle the bits using a *mixing permutation*
  - This just moves bits around
  - Goal: Spread confusion across all 128 bits
- Repeat many times

Intuition:

- Each confusion step introduces *non-linear* local changes

Confusion-Diffusion Paradigm

Many block-ciphers follow the following general approach:

- Confusion: $F_k(x) = f_1(x_1)||f_2(x_2)||\cdots||f_{16}(x_{16})$
  - $|x_i| = 8$ bits
  - f_i can be a random permutation on 8 bits
  - Goal: Introduce non-linear local changes
- Diffusion: Shuffle the bits using a *mixing permutation*
  - This just moves bits around
  - Goal: Spread confusion across all 128 bits
- Repeat many times

Intuition:

- Each confusion step introduces *non-linear* local changes
- Each diffusion step distributes those changes across blocks

Confusion-Diffusion Paradigm

Many block-ciphers follow the following general approach:

- Confusion: $F_k(x) = f_1(x_1)||f_2(x_2)||\cdots||f_{16}(x_{16})$
  - $|x_i| = 8$ bits
  - f_i can be a random permutation on 8 bits
  - Goal: Introduce non-linear local changes
- Diffusion: Shuffle the bits using a *mixing permutation*
  - This just moves bits around
  - Goal: Spread confusion across all 128 bits
- Repeat many times

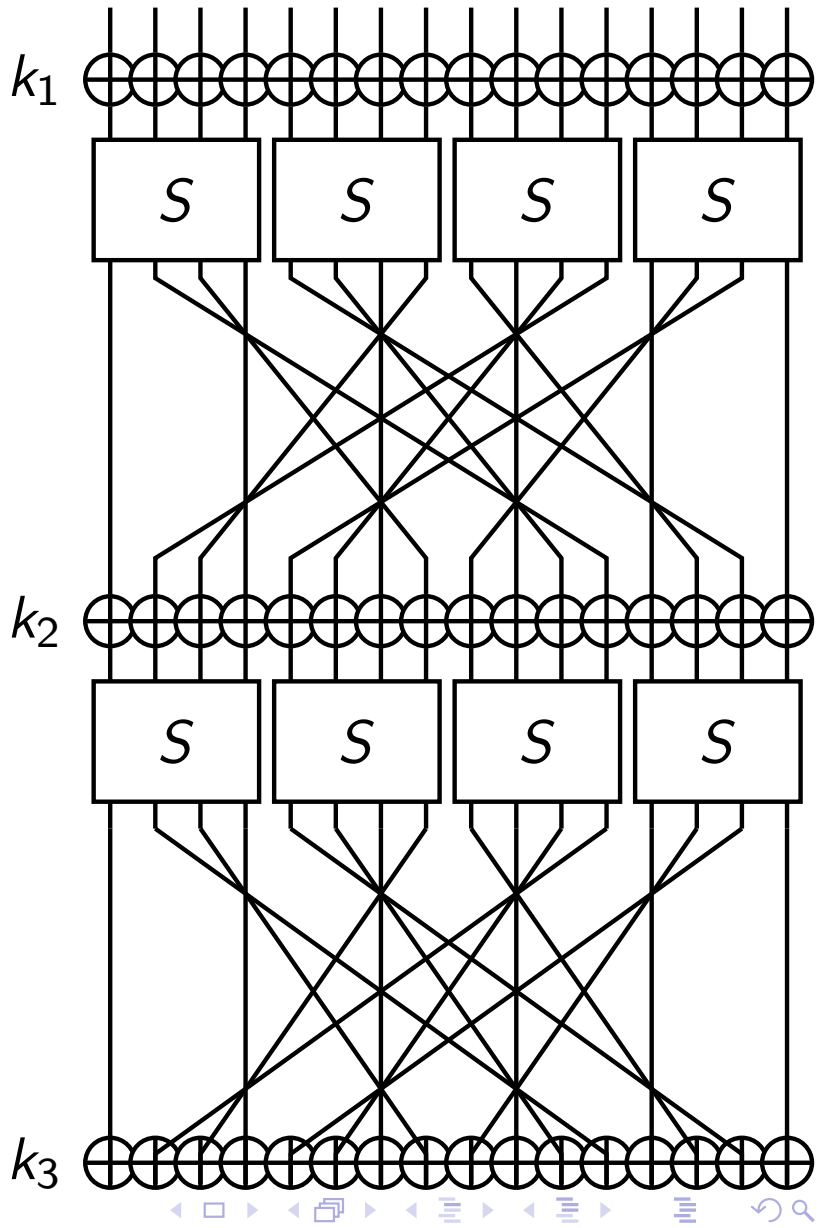
Intuition:

- Each confusion step introduces *non-linear* local changes
- Each diffusion step distributes those changes across blocks
- After many iterations, all bits are affected – avalanche effect

Outline

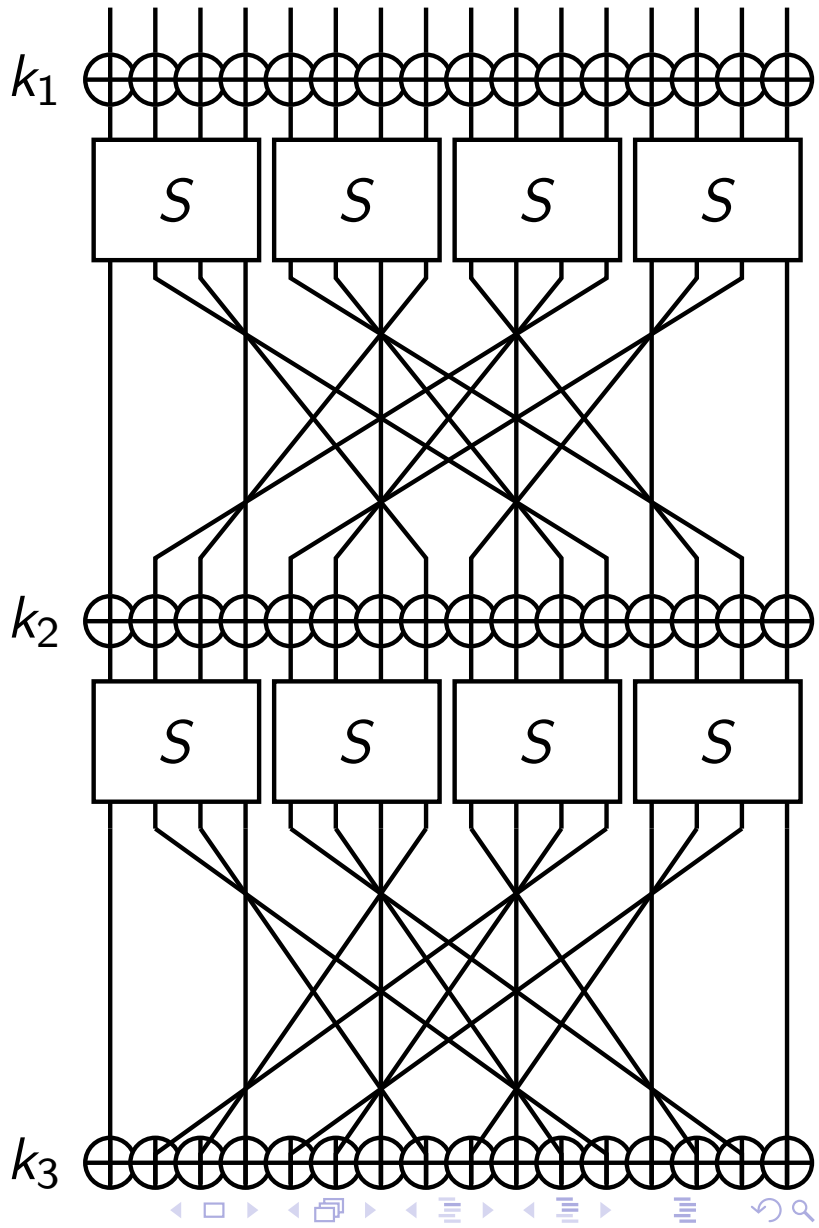
1. Constructing practical block ciphers
2. Substitution–permutation networks and AES
3. A bridge to practical hash functions
4. Sponge functions
5. Keccak, SHA-3, and SHAKE

Substitution-Permutation Network (16-bit toy example)



Substitution-Permutation Network (16-bit toy example)

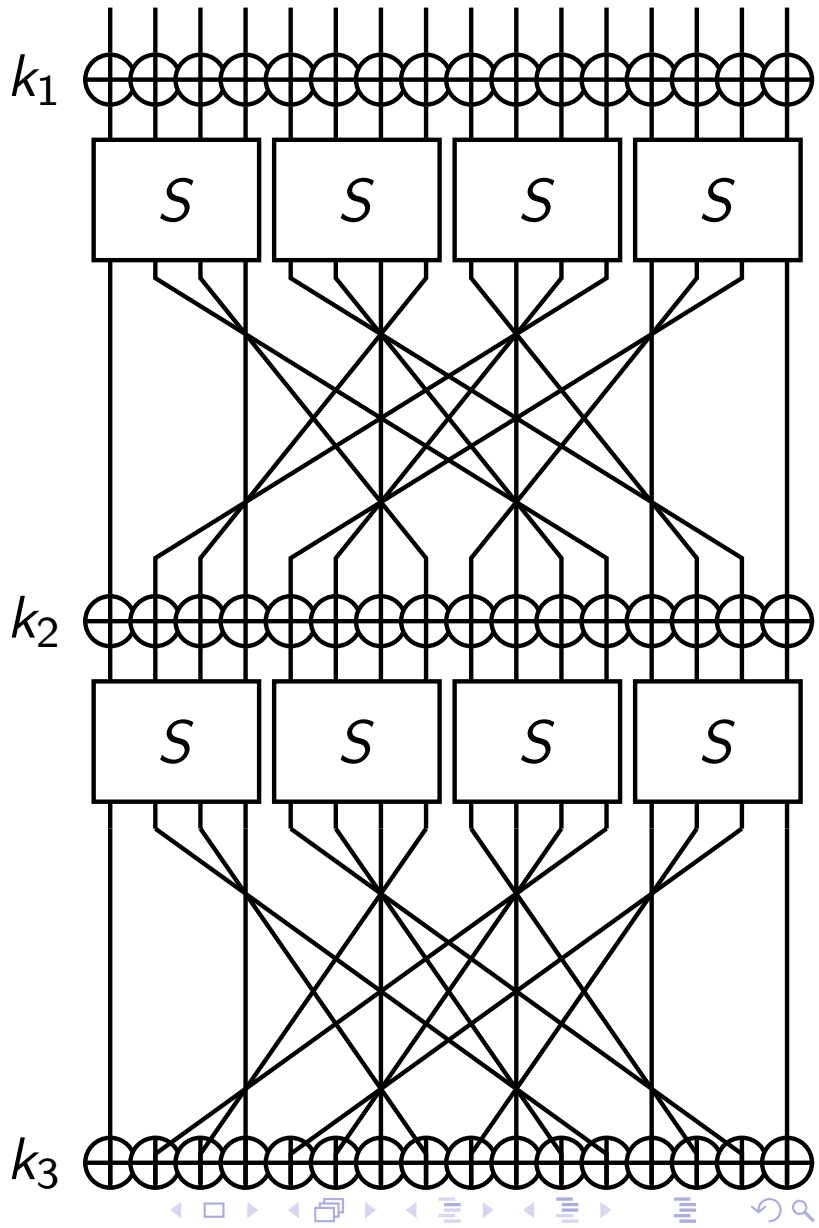
Repeat following for many rounds:



Substitution-Permutation Network (16-bit toy example)

Repeat following for many rounds:

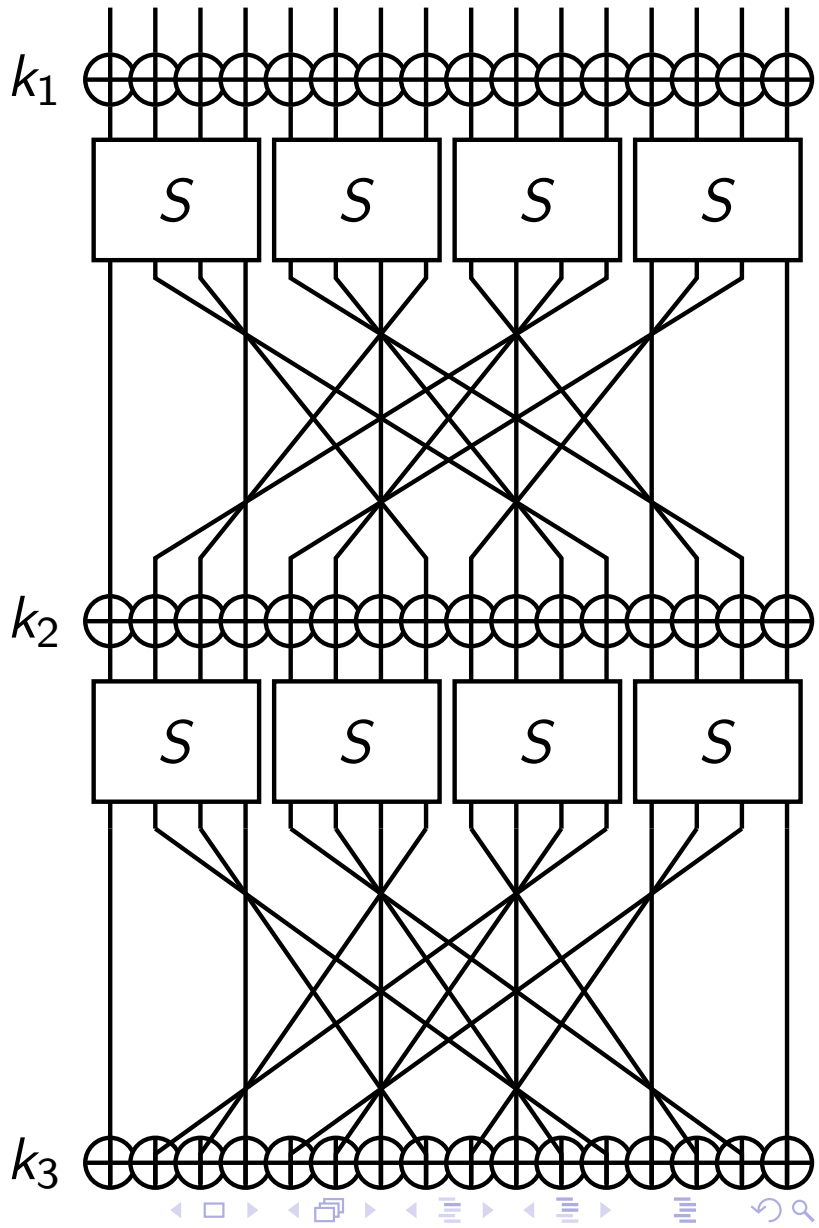
1. Key mixing: Set $x = x \oplus k_i$



Substitution-Permutation Network (16-bit toy example)

Repeat following for many rounds:

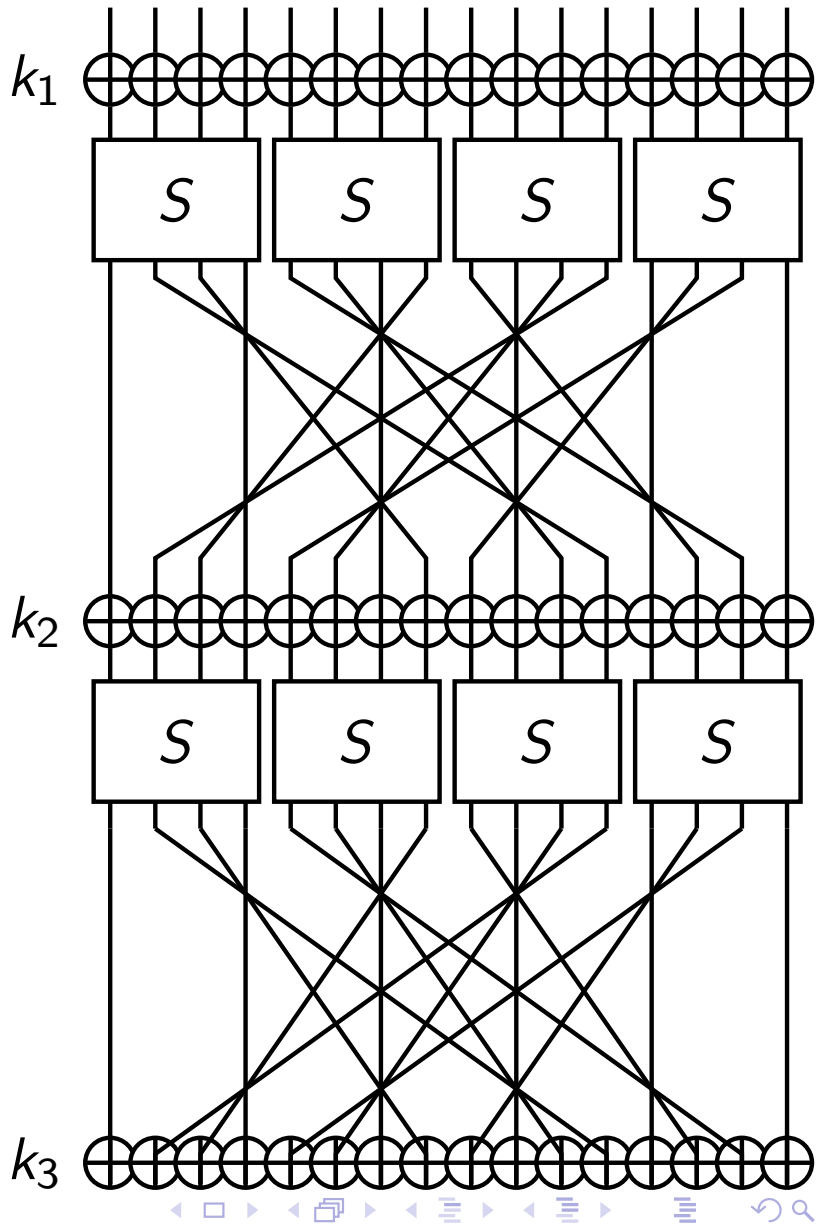
1. Key mixing: Set $x = x \oplus k_i$
2. Substitution:
   $x = S_1(x_1)||\cdots||S_4(x_4)$



Substitution-Permutation Network (16-bit toy example)

Repeat following for many rounds:

1. Key mixing: Set $x = x \oplus k_i$
2. Substitution:
   $x = S_1(x_1)||\cdots||S_4(x_4)$
3. Permutation: Permute bits of x

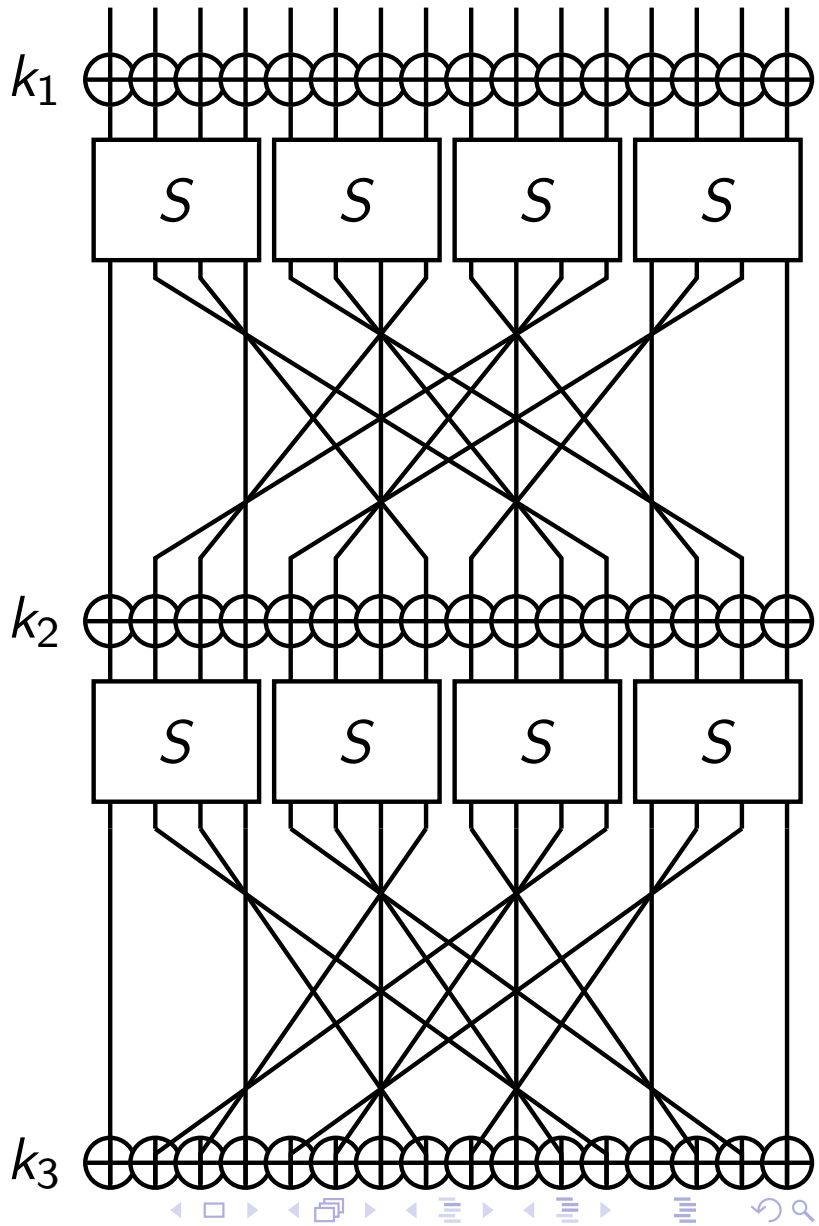


Substitution-Permutation Network (16-bit toy example)

Repeat following for many rounds:

1. Key mixing: Set $x = x \oplus k_i$
2. Substitution:
   $x = S_1(x_1)||\cdots||S_4(x_4)$
3. Permutation: Permute bits of x

Observations:



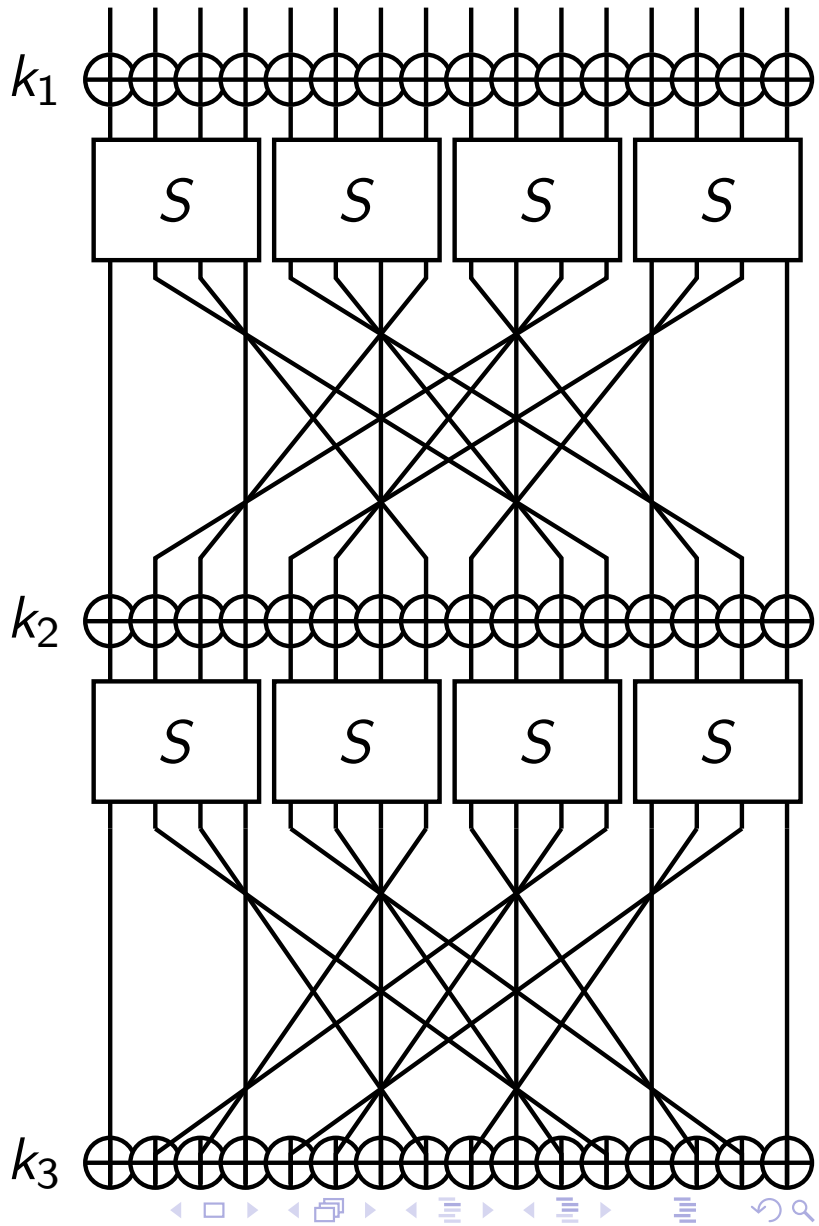
Substitution-Permutation Network (16-bit toy example)

Repeat following for many rounds:

1. Key mixing: Set $x = x \oplus k_i$
2. Substitution:
   $x = S_1(x_1)|| \cdots ||S_4(x_4)$
3. Permutation: Permute bits of x

Observations:

- $S_1, \ldots, S_4$ are public (S-boxes)



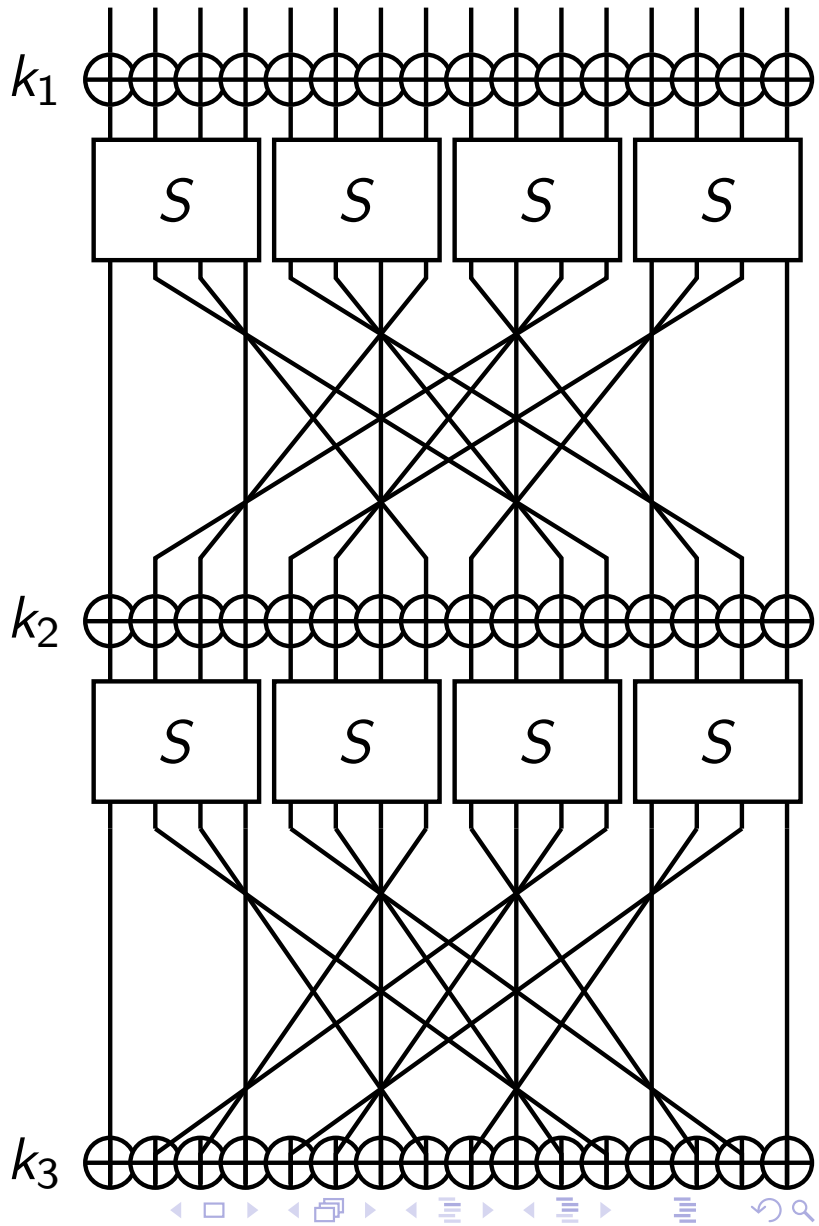
Substitution-Permutation Network (16-bit toy example)

Repeat following for many rounds:

1. Key mixing: Set $x = x \oplus k_i$
2. Substitution:
   $x = S_1(x_1)||\cdots||S_4(x_4)$
3. Permutation: Permute bits of x

Observations:

- $S_1, \ldots, S_4$ are public (S-boxes)
- Mixing perm. is also public



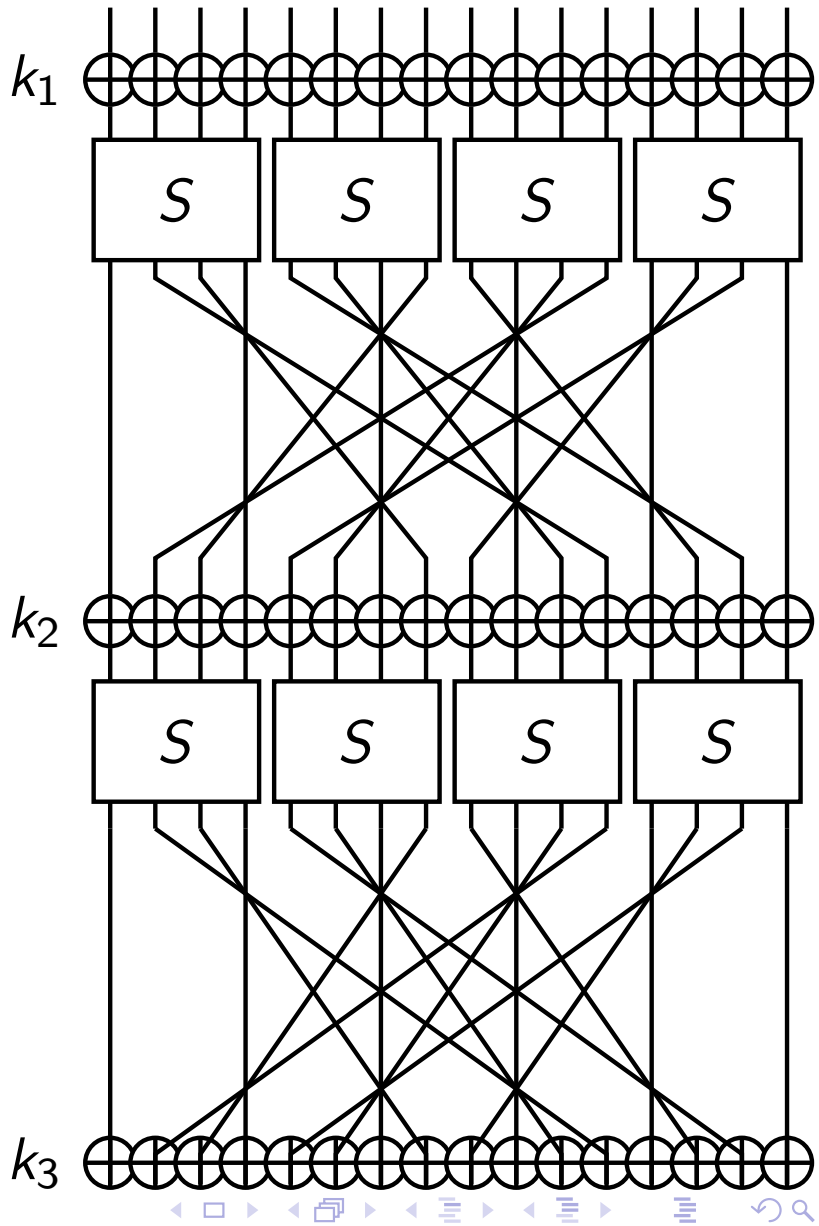
Substitution-Permutation Network (16-bit toy example)

Repeat following for many rounds:

1. Key mixing: Set $x = x \oplus k_i$
2. Substitution:
   $x = S_1(x_1)|| \cdots ||S_4(x_4)$
3. Permutation: Permute bits of x

Observations:

- $S_1, \ldots, S_4$ are public (S-boxes)
- Mixing perm. is also public
- Key k only used in key-mixing



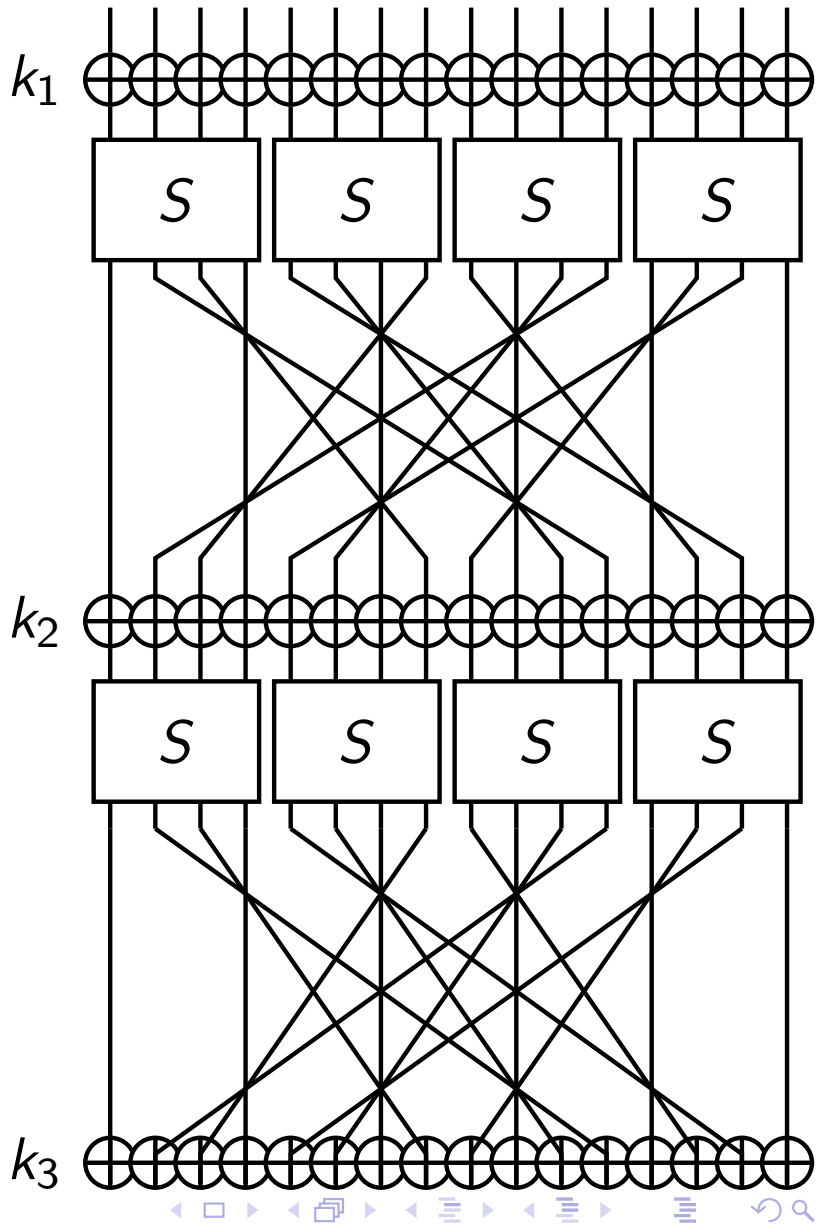
Substitution-Permutation Network (16-bit toy example)

Repeat following for many rounds:

1. Key mixing: Set $x = x \oplus k_i$
2. Substitution: $x = S_1(x_1)||\cdots||S_4(x_4)$
3. Permutation: Permute bits of x

Observations:

- $S_1, \ldots, S_4$ are public (S-boxes)
- Mixing perm. is also public
- Key k only used in key-mixing
- Need one more key-mixing step after last round to prevent $\mathcal{A}$ from peeling off last mixing layer



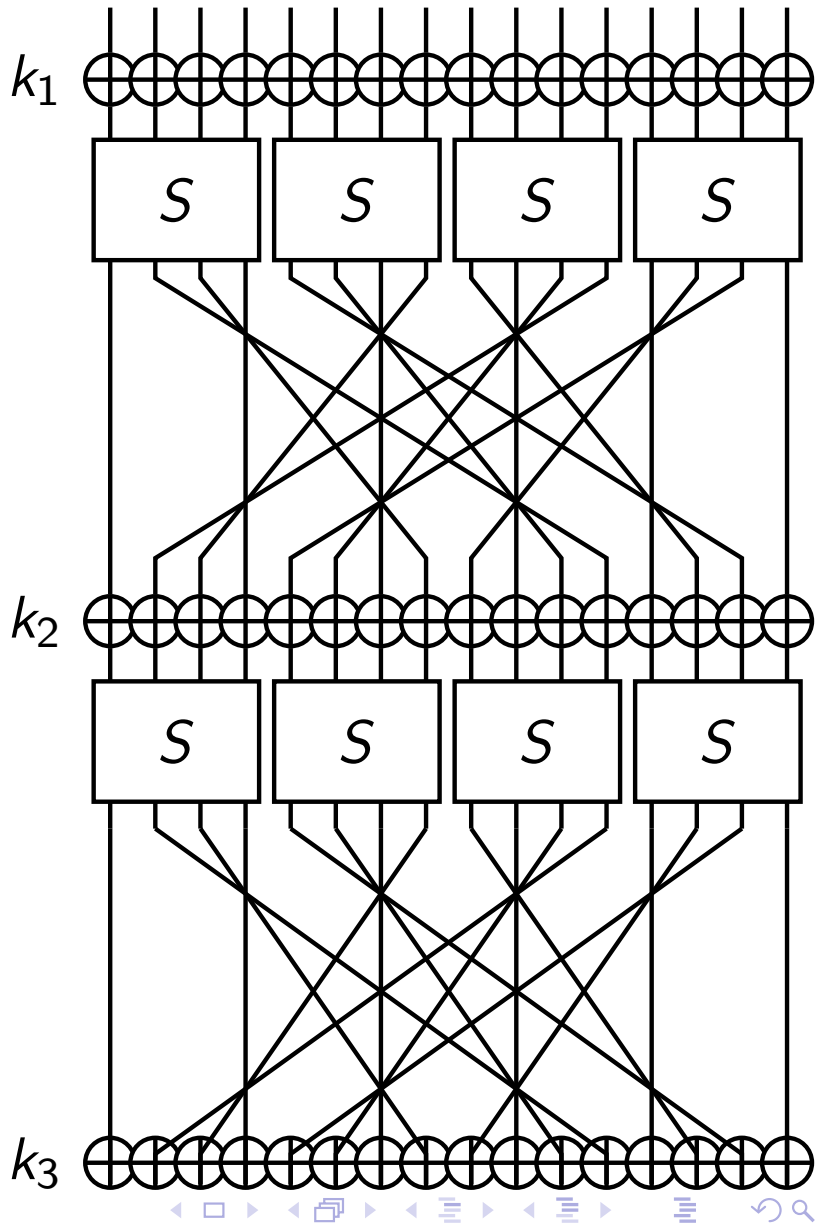
Substitution-Permutation Network (16-bit toy example)

Repeat following for many rounds:

1. Key mixing: Set $x = x \oplus k_i$
2. Substitution: $x = S_1(x_1)||\cdots||S_4(x_4)$
3. Permutation: Permute bits of x

Observations:

- $S_1, \ldots, S_4$ are public (S-boxes)
- Mixing perm. is also public
- Key k only used in key-mixing
- Need one more key-mixing step after last round to prevent $\mathcal{A}$ from peeling off last mixing layer
- Direct application of confusion-diffusion paradigm

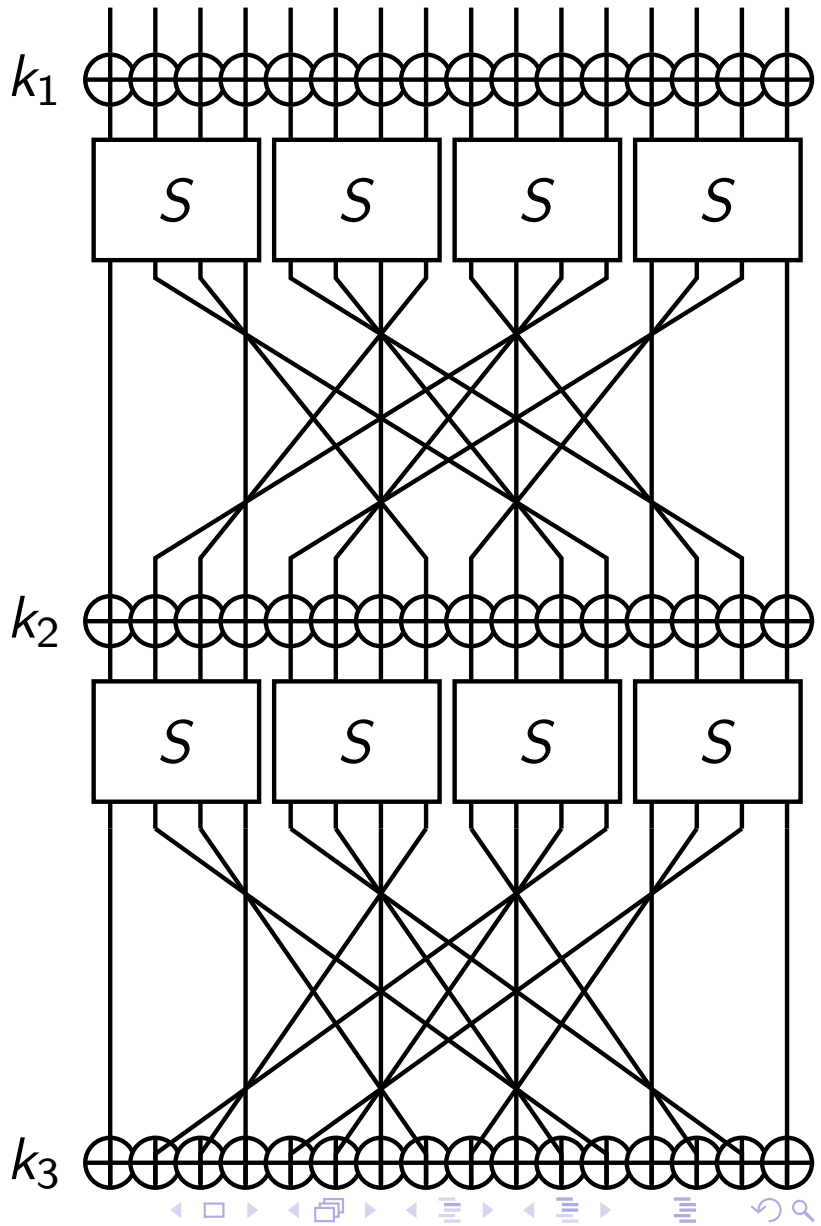


Substitution-Permutation Network (16-bit toy example)

Repeat following for many rounds:

1. Key mixing: Set $x = x \oplus k_i$
2. Substitution:
   $x = S_1(x_1)||\cdots||S_4(x_4)$
3. Permutation: Permute bits of x

Key Schedule:



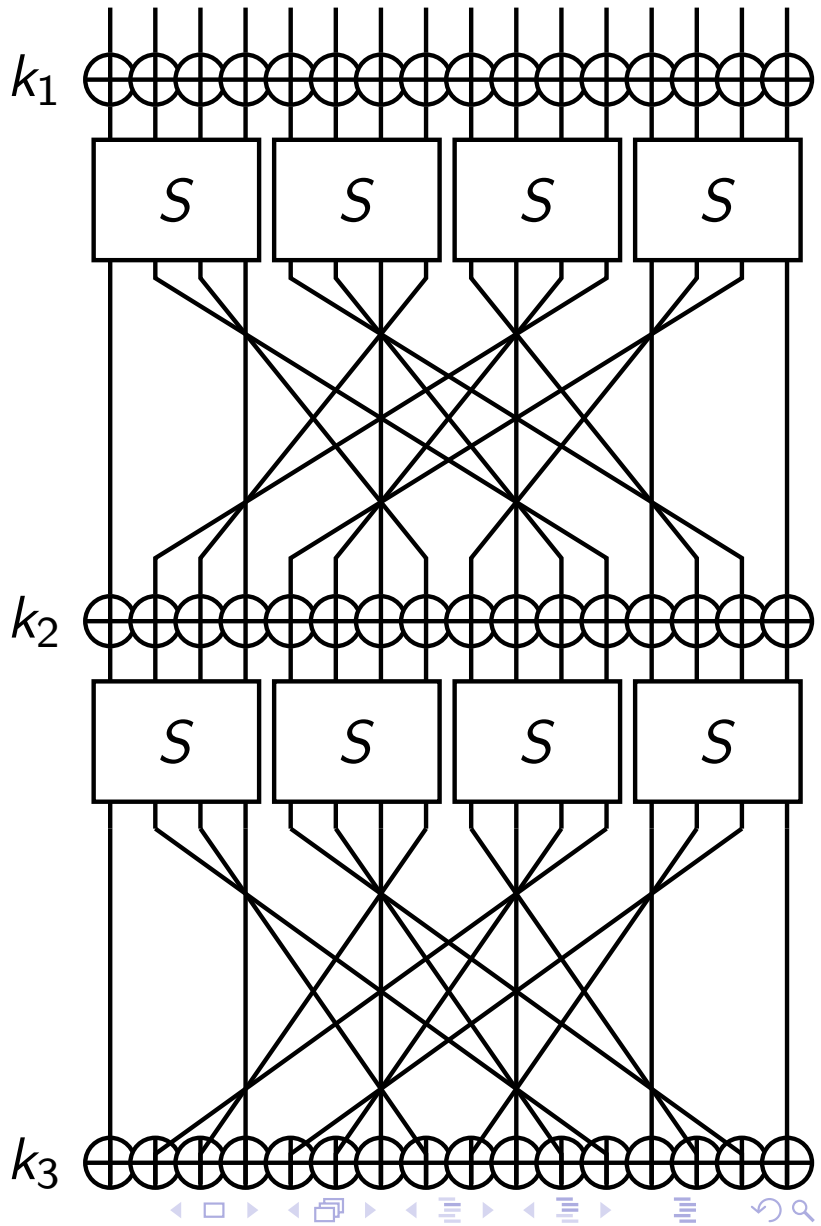
Substitution-Permutation Network (16-bit toy example)

Repeat following for many rounds:

1. Key mixing: Set $x = x \oplus k_i$
2. Substitution:
   $x = S_1(x_1)||\cdots||S_4(x_4)$
3. Permutation: Permute bits of x

Key Schedule:

- Each round uses different key k_i



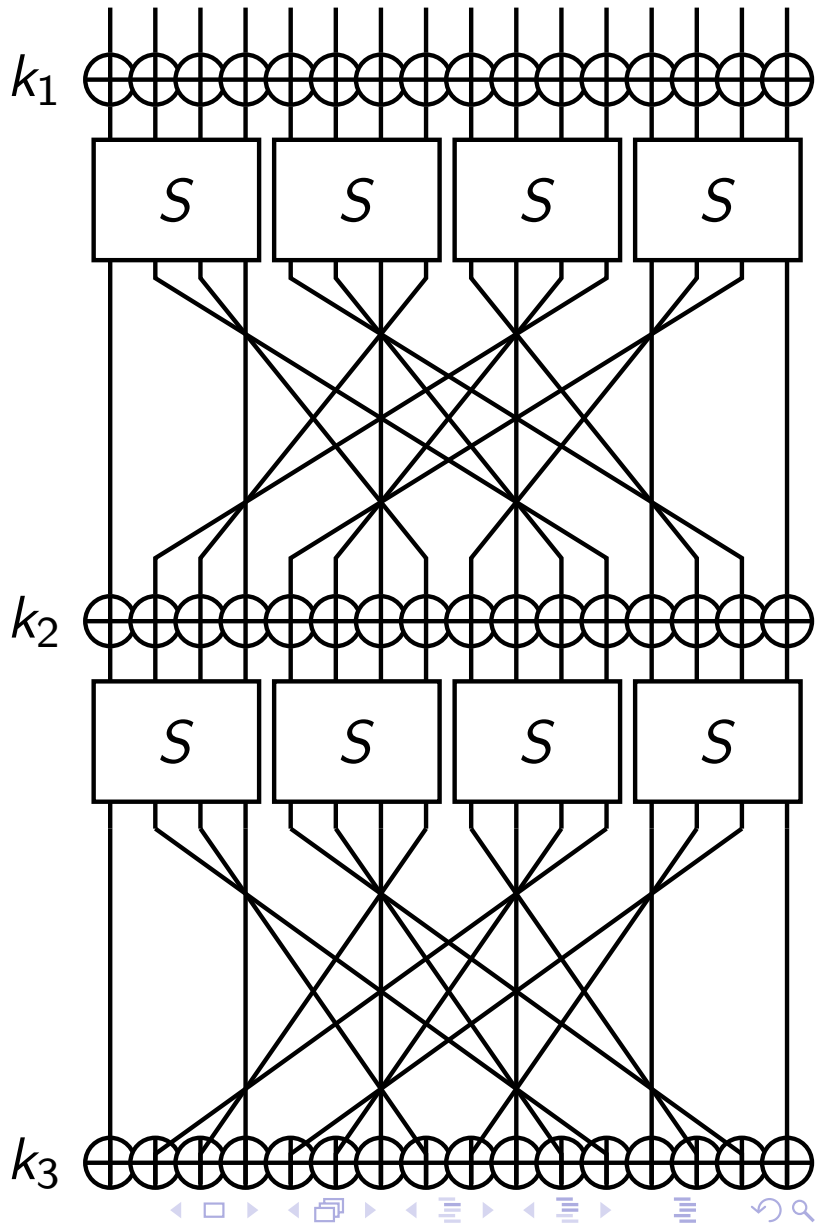
Substitution-Permutation Network (16-bit toy example)

Repeat following for many rounds:

1. Key mixing: Set $x = x \oplus k_i$
2. Substitution: $x = S_1(x_1)||\cdots||S_4(x_4)$
3. Permutation: Permute bits of x

Key Schedule:

- Each round uses different key k_i
- k_i are derived from *master key* k via *key schedule*



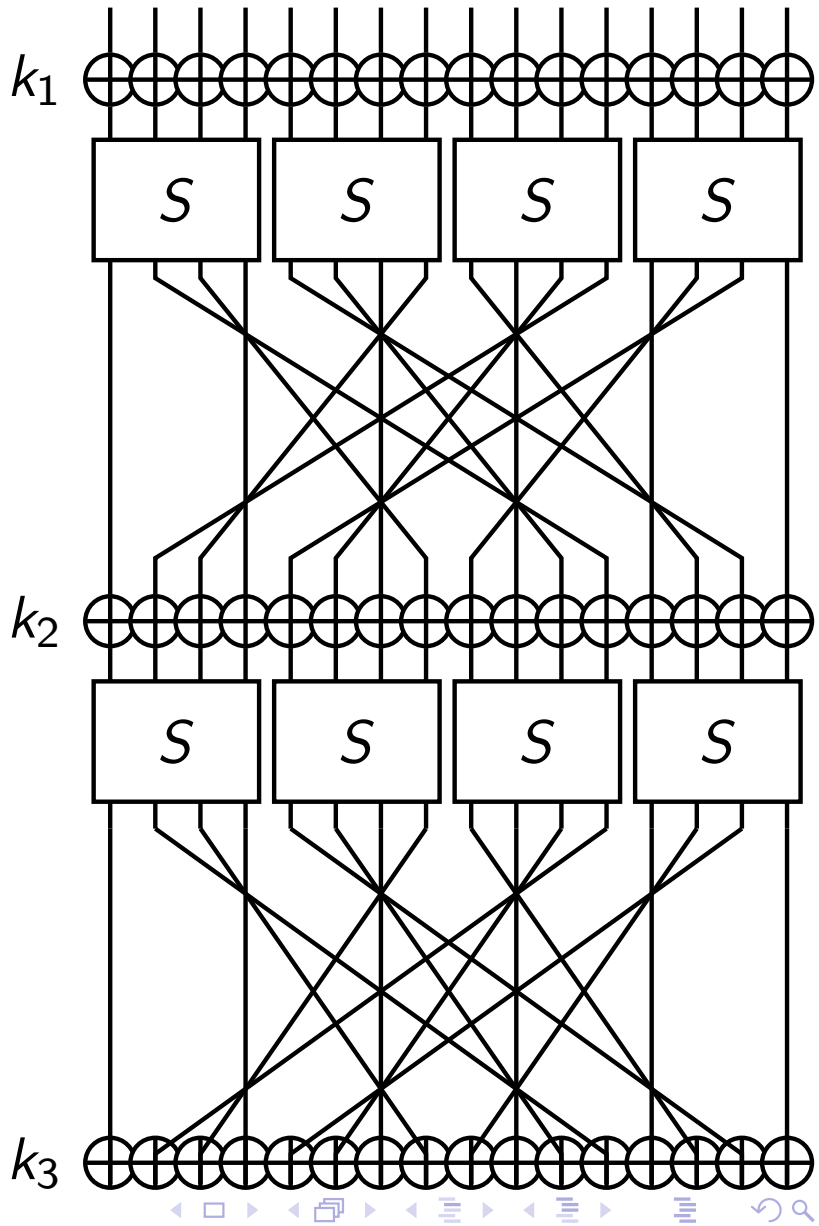
Substitution-Permutation Network (16-bit toy example)

Repeat following for many rounds:

1. Key mixing: Set $x = x \oplus k_i$
2. Substitution: $x = S_1(x_1)||\cdots||S_4(x_4)$
3. Permutation: Permute bits of x

Key Schedule:

- Each round uses different key k_i
- k_i are derived from *master key* k via *key schedule*
- The key schedule expands k into round keys; these keys are not independent



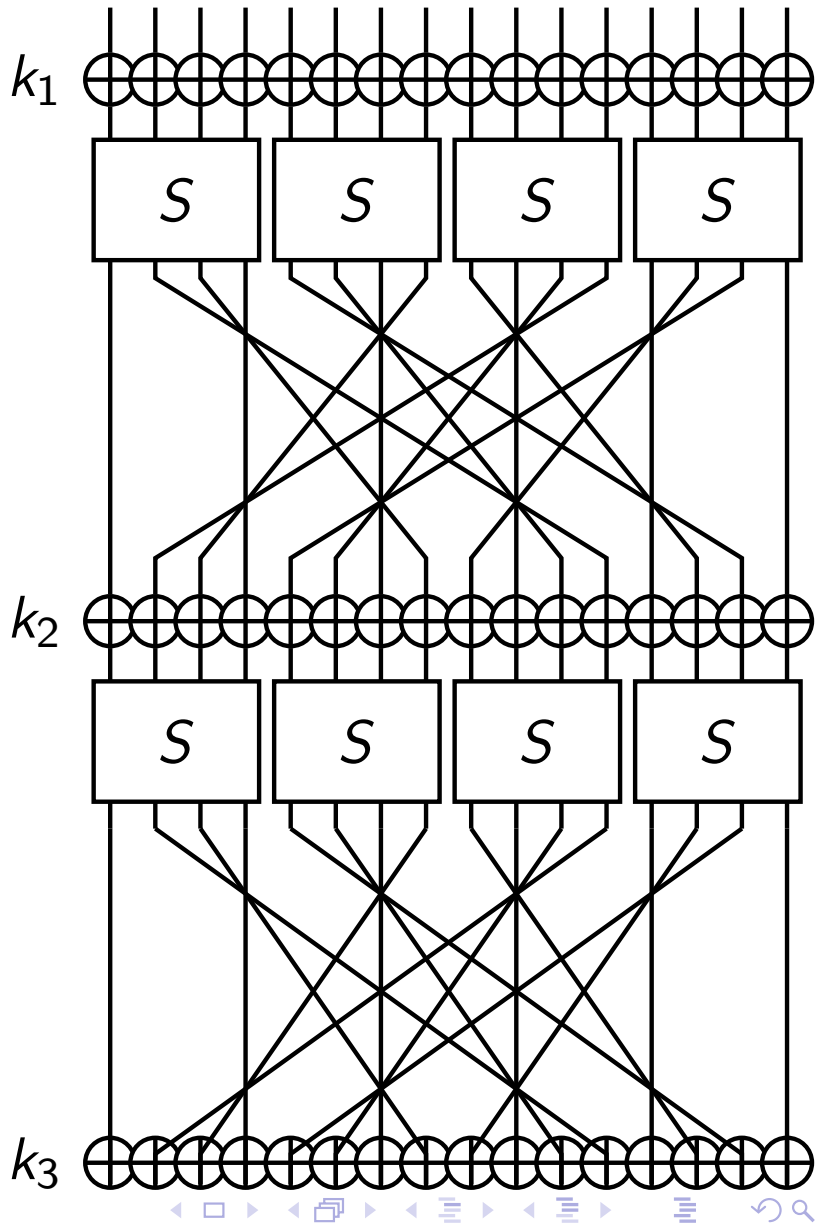
Substitution-Permutation Network (16-bit toy example)

Repeat following for many rounds:

1. Key mixing: Set $x = x \oplus k_i$
2. Substitution: $x = S_1(x_1)||\cdots||S_4(x_4)$
3. Permutation: Permute bits of x

Key Schedule:

- Each round uses different key k_i
- k_i are derived from *master key* k via *key schedule*
- The key schedule expands k into round keys; these keys are not independent
- r-round SPN needs $r + 1$ keys



AES / Rijndael

- Designed by Joan Daemen and Vincent Rijmen.
- Rijndael was selected in the NIST competition in 2000; AES was standardized in 2001.
- AES fixes the block length at 128 bits.
- Hardware instructions can accelerate the round transformations.
- We will examine the state and the role of each transformation.

AES / Rijndael

- Designed by Joan Daemen and Vincent Rijmen.
- Rijndael was selected in the NIST competition in 2000; AES was standardized in 2001.
- AES fixes the block length at 128 bits.
- Hardware instructions can accelerate the round transformations.
- We will examine the state and the role of each transformation.

Connection to our earlier constructions

AES can instantiate the block cipher in CBC or CTR, or in authenticated-encryption modes such as GCM.

AES Parameters and State

Available Versions: $\ell = 128(16 \text{ Bytes}), n = 128, 192, 256$

AES Parameters and State

Available Versions: $\ell = 128(16 \text{ Bytes}), n = 128, 192, 256$

- Change key schedule for different key lengths

AES Parameters and State

Available Versions: $\ell = 128(16 \text{ Bytes}), n = 128, 192, 256$

- Change key schedule for different key lengths
- 10 rounds for 128-bit key, 12 rounds for 192-bit key, 14 rounds for 256-bit key

AES Parameters and State

Available Versions: $\ell = 128(16 \text{ Bytes}), n = 128, 192, 256$

- Change key schedule for different key lengths
- 10 rounds for 128-bit key, 12 rounds for 192-bit key, 14 rounds for 256-bit key

State: 4×4 array of Bytes

$$\begin{pmatrix} B_{11} & B_{12} & B_{13} & B_{14} \\ B_{21} & B_{22} & B_{23} & B_{24} \\ B_{31} & B_{32} & B_{33} & B_{34} \\ B_{41} & B_{42} & B_{43} & B_{44} \end{pmatrix}$$

AES: Key Addition and Substitution

1. Add Round Key: View $k_i = k_{11}||k_{12}||\cdots||k_{44}$ with $|k_{ij}| = 1$ Byte

$$\begin{pmatrix} B_{11} \oplus k_{11} & B_{12} \oplus k_{12} & B_{13} \oplus k_{13} & B_{14} \oplus k_{14} \\ B_{21} \oplus k_{21} & B_{22} \oplus k_{22} & B_{23} \oplus k_{23} & B_{24} \oplus k_{24} \\ B_{31} \oplus k_{31} & B_{32} \oplus k_{32} & B_{33} \oplus k_{33} & B_{34} \oplus k_{34} \\ B_{41} \oplus k_{41} & B_{42} \oplus k_{42} & B_{43} \oplus k_{43} & B_{44} \oplus k_{44} \end{pmatrix}$$

AES: Key Addition and Substitution

1. Add Round Key: View $k_i = k_{11}||k_{12}||\cdots||k_{44}$ with $|k_{ij}| = 1$ Byte

$$\begin{pmatrix} B_{11} \oplus k_{11} & B_{12} \oplus k_{12} & B_{13} \oplus k_{13} & B_{14} \oplus k_{14} \\ B_{21} \oplus k_{21} & B_{22} \oplus k_{22} & B_{23} \oplus k_{23} & B_{24} \oplus k_{24} \\ B_{31} \oplus k_{31} & B_{32} \oplus k_{32} & B_{33} \oplus k_{33} & B_{34} \oplus k_{34} \\ B_{41} \oplus k_{41} & B_{42} \oplus k_{42} & B_{43} \oplus k_{43} & B_{44} \oplus k_{44} \end{pmatrix}$$

2. SubBytes: Uses a single 8-bit S-box

$$\begin{pmatrix} S(B_{11}) & S(B_{12}) & S(B_{13}) & S(B_{14}) \\ S(B_{21}) & S(B_{22}) & S(B_{23}) & S(B_{24}) \\ S(B_{31}) & S(B_{32}) & S(B_{33}) & S(B_{34}) \\ S(B_{41}) & S(B_{42}) & S(B_{43}) & S(B_{44}) \end{pmatrix}$$

AES: Spreading Changes Across the State

3. Shift Rows: Shift each row to the left by varying amounts

$$\begin{pmatrix} B_{11} & B_{12} & B_{13} & B_{14} & (0 \text{ shift}) \\ B_{22} & B_{23} & B_{24} & B_{21} & (1 \text{ shift}) \\ B_{33} & B_{34} & B_{31} & B_{32} & (2 \text{ shift}) \\ B_{44} & B_{41} & B_{42} & B_{43} & (3 \text{ shift}) \end{pmatrix}$$

AES: Spreading Changes Across the State

3. Shift Rows: Shift each row to the left by varying amounts

$$\begin{pmatrix} B_{11} & B_{12} & B_{13} & B_{14} & (0 \text{ shift}) \\ B_{22} & B_{23} & B_{24} & B_{21} & (1 \text{ shift}) \\ B_{33} & B_{34} & B_{31} & B_{32} & (2 \text{ shift}) \\ B_{44} & B_{41} & B_{42} & B_{43} & (3 \text{ shift}) \end{pmatrix}$$

4. MixColumns: Apply invertible transformation to Bytes in each column – change in 1 Byte affects all 4 Bytes in that column

AES: Spreading Changes Across the State

3. Shift Rows: Shift each row to the left by varying amounts

$$\begin{pmatrix} B_{11} & B_{12} & B_{13} & B_{14} & (0 \text{ shift}) \\ B_{22} & B_{23} & B_{24} & B_{21} & (1 \text{ shift}) \\ B_{33} & B_{34} & B_{31} & B_{32} & (2 \text{ shift}) \\ B_{44} & B_{41} & B_{42} & B_{43} & (3 \text{ shift}) \end{pmatrix}$$

4. MixColumns: Apply invertible transformation to Bytes in each column – change in 1 Byte affects all 4 Bytes in that column

Observations:

AES: Spreading Changes Across the State

3. Shift Rows: Shift each row to the left by varying amounts

$$\begin{pmatrix} B_{11} & B_{12} & B_{13} & B_{14} & (0 \text{ shift}) \\ B_{22} & B_{23} & B_{24} & B_{21} & (1 \text{ shift}) \\ B_{33} & B_{34} & B_{31} & B_{32} & (2 \text{ shift}) \\ B_{44} & B_{41} & B_{42} & B_{43} & (3 \text{ shift}) \end{pmatrix}$$

4. MixColumns: Apply invertible transformation to Bytes in each column – change in 1 Byte affects all 4 Bytes in that column

Observations:

- ShiftRows moves bytes between columns; MixColumns mixes bytes within each column

AES: Spreading Changes Across the State

3. Shift Rows: Shift each row to the left by varying amounts

$$\begin{pmatrix} B_{11} & B_{12} & B_{13} & B_{14} & (0 \text{ shift}) \\ B_{22} & B_{23} & B_{24} & B_{21} & (1 \text{ shift}) \\ B_{33} & B_{34} & B_{31} & B_{32} & (2 \text{ shift}) \\ B_{44} & B_{41} & B_{42} & B_{43} & (3 \text{ shift}) \end{pmatrix}$$

4. MixColumns: Apply invertible transformation to Bytes in each column – change in 1 Byte affects all 4 Bytes in that column

Observations:

- ShiftRows moves bytes between columns; MixColumns mixes bytes within each column
- Both operations are invertible; MixColumns combines bytes, rather than merely rearranging them

Putting the AES Rounds in Order

AES-128 encryption: 10 rounds

1. Initial **AddRoundKey**.
2. Rounds 1–9: **SubBytes** $\rightarrow$ **ShiftRows** $\rightarrow$ **MixColumns** $\rightarrow$ **AddRoundKey**.
3. Round 10: **SubBytes** $\rightarrow$ **ShiftRows** $\rightarrow$ **AddRoundKey**.

Putting the AES Rounds in Order

AES-128 encryption: 10 rounds

1. Initial **AddRoundKey**.
2. Rounds 1–9: **SubBytes** $\rightarrow$ **ShiftRows** $\rightarrow$ **MixColumns** $\rightarrow$ **AddRoundKey**.
3. Round 10: **SubBytes** $\rightarrow$ **ShiftRows** $\rightarrow$ **AddRoundKey**.

- The final round omits MixColumns.
- The key schedule derives 11 round keys from the master key.
- All steps are invertible: decryption applies inverses in reverse order.
- AES-192 and AES-256 use 12 and 14 rounds, respectively.

Exercise: Follow a Difference Through AES

Two states differ in exactly one byte. Apply one ordinary AES round using the same round key.

1. After SubBytes, how many byte positions differ?

Exercise: Follow a Difference Through AES

Two states differ in exactly one byte. Apply one ordinary AES round using the same round key.

1. After SubBytes, how many byte positions differ?
2. After ShiftRows, where does that difference go?

Exercise: Follow a Difference Through AES

Two states differ in exactly one byte. Apply one ordinary AES round using the same round key.

1. After SubBytes, how many byte positions differ?
2. After ShiftRows, where does that difference go?
3. After MixColumns, how many bytes differ?

Exercise: Follow a Difference Through AES

Two states differ in exactly one byte. Apply one ordinary AES round using the same round key.

1. After SubBytes, how many byte positions differ?
2. After ShiftRows, where does that difference go?
3. After MixColumns, how many bytes differ?
4. What happens after the next round's ShiftRows and MixColumns?

Discussion

What would go wrong if we removed ShiftRows from every round?

Following the Difference: Discussion

- SubBytes is a byte permutation: the one differing byte remains different.

Following the Difference: Discussion

- SubBytes is a byte permutation: the one differing byte remains different.
- ShiftRows relocates it; the difference is still in one byte.

Following the Difference: Discussion

- SubBytes is a byte permutation: the one differing byte remains different.
- ShiftRows relocates it; the difference is still in one byte.
- MixColumns spreads this single-byte difference to all four bytes of its column.

Following the Difference: Discussion

- SubBytes is a byte permutation: the one differing byte remains different.
- ShiftRows relocates it; the difference is still in one byte.
- MixColumns spreads this single-byte difference to all four bytes of its column.
- Next round: ShiftRows puts these four differences in distinct columns; MixColumns spreads them to all 16 bytes.

Following the Difference: Discussion

- SubBytes is a byte permutation: the one differing byte remains different.
- ShiftRows relocates it; the difference is still in one byte.
- MixColumns spreads this single-byte difference to all four bytes of its column.
- Next round: ShiftRows puts these four differences in distinct columns; MixColumns spreads them to all 16 bytes.

Without ShiftRows

Columns would evolve independently. An attacker could test these separations instead of seeing one well-mixed 128-bit permutation.

Following the Difference: Discussion

- SubBytes is a byte permutation: the one differing byte remains different.
- ShiftRows relocates it; the difference is still in one byte.
- MixColumns spreads this single-byte difference to all four bytes of its column.
- Next round: ShiftRows puts these four differences in distinct columns; MixColumns spreads them to all 16 bytes.

Without ShiftRows

Columns would evolve independently. An attacker could test these separations instead of seeing one well-mixed 128-bit permutation.

This explains diffusion, not a security proof for full AES.

What AES Gives Us—and What the Mode Must Add

- AES: fixed-size keyed permutation; deterministic for a fixed key.
- CTR: turns block-cipher outputs into a pad; counter inputs must not repeat under a key.
- CBC: chains blocks; CPA security requires an unpredictable IV.
- Encryption alone does not provide integrity.
- Authenticated encryption combines confidentiality and integrity; nonce and tag handling still matter.

Takeaway

A strong primitive does not repair an insecure composition or incorrect use.

Break

We have a practical keyed permutation.

How can we build a hash function?

Outline

1. Constructing practical block ciphers
2. Substitution–permutation networks and AES
3. A bridge to practical hash functions
4. Sponge functions
5. Keccak, SHA-3, and SHAKE

From Block Ciphers to Hash Functions

- AES showed how small operations and repeated rounds build a transformation on fixed-size blocks.

From Block Ciphers to Hash Functions

- AES showed how small operations and repeated rounds build a transformation on fixed-size blocks.
- A hash must handle an arbitrarily long message and produce a fixed-size digest.

From Block Ciphers to Hash Functions

- AES showed how small operations and repeated rounds build a transformation on fixed-size blocks.
- A hash must handle an arbitrarily long message and produce a fixed-size digest.
- Idea: process one message block at a time, maintaining a fixed-size running state.

From Block Ciphers to Hash Functions

- AES showed how small operations and repeated rounds build a transformation on fixed-size blocks.
- A hash must handle an arbitrarily long message and produce a fixed-size digest.
- Idea: process one message block at a time, maintaining a fixed-size running state.

Two design questions

1. What fixed-size operation updates the state?

From Block Ciphers to Hash Functions

- AES showed how small operations and repeated rounds build a transformation on fixed-size blocks.
- A hash must handle an arbitrarily long message and produce a fixed-size digest.
- Idea: process one message block at a time, maintaining a fixed-size running state.

Two design questions

1. What fixed-size operation updates the state?
2. How do we combine those updates into a hash of the whole message?

A Compression Function Updates the State

$$C : \{0,1\}^n \times \{0,1\}^b \to \{0,1\}^n, \qquad (h, M) \longmapsto h'.$$

- Input: an n-bit chaining value h and a b-bit message block M.
- Output: an updated n-bit chaining value h'.

A Compression Function Updates the State

$$C : \{0,1\}^n \times \{0,1\}^b \to \{0,1\}^n, \qquad (h, M) \longmapsto h'.$$

- Input: an n-bit chaining value h and a b-bit message block M.
- Output: an updated n-bit chaining value h'.
- Observation: Like a hash function but with a fixed-size input.

A Compression Function Updates the State

$$C : \{0,1\}^n \times \{0,1\}^b \to \{0,1\}^n, \qquad (h, M) \longmapsto h'.$$

- Input: an n-bit chaining value h and a b-bit message block M.
- Output: an updated n-bit chaining value h'.
- Observation: Like a hash function but with a fixed-size input.
- Like block ciphers, compression functions can use nonlinear operations, mixing, and repeated rounds; they can also be constructed from block ciphers.

A Compression Function Updates the State

$$C : \{0,1\}^n \times \{0,1\}^b \to \{0,1\}^n, \qquad (h, M) \longmapsto h'.$$

- Input: an n-bit chaining value h and a b-bit message block M.
- Output: an updated n-bit chaining value h'.
- Observation: Like a hash function but with a fixed-size input.
- Like block ciphers, compression functions can use nonlinear operations, mixing, and repeated rounds; they can also be constructed from block ciphers.
- Unlike a block cipher, C is public and compresses its combined input. We seek collision resistance, rather than a secret keyed permutation.

A Compression Function Updates the State

$$C : \{0,1\}^n \times \{0,1\}^b \to \{0,1\}^n, \qquad (h, M) \longmapsto h'.$$

- Input: an n-bit chaining value h and a b-bit message block M.
- Output: an updated n-bit chaining value h'.
- Observation: Like a hash function but with a fixed-size input.
- Like block ciphers, compression functions can use nonlinear operations, mixing, and repeated rounds; they can also be constructed from block ciphers.
- Unlike a block cipher, C is public and compresses its combined input. We seek collision resistance, rather than a secret keyed permutation.

Next: Domain extension

Given this fixed-size component, how do we hash an arbitrary-length message?

Iterating a Compression Function – Domain Extension

Merkle–Damgård: the high-level picture

Assume m has been unambiguously encoded and padded into full blocks $M_1, \ldots, M_t$.

$$h_0 = \mathrm{IV}, \qquad h_i = C(h_{i-1}, M_i), \qquad H(m) = h_t.$$

- C maps a chaining value and one message block to a new chaining value.

Iterating a Compression Function – Domain Extension

Merkle–Damgård: the high-level picture

Assume m has been unambiguously encoded and padded into full blocks $M_1, \ldots, M_t$.

$$h_0 = \mathrm{IV}, \qquad h_i = C(h_{i-1}, M_i), \qquad H(m) = h_t.$$

- C maps a chaining value and one message block to a new chaining value.
- A fixed-size component handles arbitrary-length messages.
- Collision resistance of C transfers to H.

Iterating a Compression Function – Domain Extension

Merkle–Damgård: the high-level picture

Assume m has been unambiguously encoded and padded into full blocks $M_1, \ldots, M_t$.

$$h_0 = \mathrm{IV}, \qquad h_i = C(h_{i-1}, M_i), \qquad H(m) = h_t.$$

- C maps a chaining value and one message block to a new chaining value.
- A fixed-size component handles arbitrary-length messages.
- Collision resistance of C transfers to H.
- SHA-256 follows this general pattern, using a dedicated compression function.

Why the Shape of the Construction Matters

For the basic construction that outputs the full chaining state:

$$H(m) = h_t \quad \Longrightarrow \quad \text{attacker can continue the iteration.}$$

$$h_{t+1} = C(h_t, M_{t+1}), \quad h_{t+2} = C(h_{t+1}, M_{t+2}), \ldots$$

Why the Shape of the Construction Matters

For the basic construction that outputs the full chaining state:

$$H(m) = h_t \quad \Longrightarrow \quad \text{attacker can continue the iteration.}$$

$$h_{t+1} = C(h_t, M_{t+1}), \quad h_{t+2} = C(h_{t+1}, M_{t+2}), \ldots$$

- Given $H(m)$ and $|m|$, compute $H(m \| \mathrm{pad}(m) \| x)$ without knowing m.

Why the Shape of the Construction Matters

For the basic construction that outputs the full chaining state:

$$H(m) = h_t \quad \Longrightarrow \quad \text{attacker can continue the iteration.}$$

$$h_{t+1} = C(h_t, M_{t+1}), \quad h_{t+2} = C(h_{t+1}, M_{t+2}), \ldots$$

- Given $H(m)$ and $|m|$, compute $H(m \| \mathrm{pad}(m) \| x)$ without knowing m.
- Here $\mathrm{pad}(m)$ is the original padding: the attacker extends the already-padded message, not simply m.

Why the Shape of the Construction Matters

For the basic construction that outputs the full chaining state:

$$H(m) = h_t \quad \Longrightarrow \quad \text{attacker can continue the iteration.}$$

$$h_{t+1} = C(h_t, M_{t+1}), \quad h_{t+2} = C(h_{t+1}, M_{t+2}), \ldots$$

- Given $H(m)$ and $|m|$, compute $H(m \| \mathrm{pad}(m) \| x)$ without knowing m.
- Here $\mathrm{pad}(m)$ is the original padding: the attacker extends the already-padded message, not simply m.
- This is **length extension**; it does not produce a collision.

Why the Shape of the Construction Matters

For the basic construction that outputs the full chaining state:

$$H(m) = h_t \quad \Longrightarrow \quad \text{attacker can continue the iteration.}$$

$$h_{t+1} = C(h_t, M_{t+1}), \quad h_{t+2} = C(h_{t+1}, M_{t+2}), \ldots$$

- Given $H(m)$ and $|m|$, compute $H(m \| \mathrm{pad}(m) \| x)$ without knowing m.
- Here $\mathrm{pad}(m)$ is the original padding: the attacker extends the already-padded message, not simply m.
- This is **length extension**; it does not produce a collision.
- It can break naive secret-prefix authentication $H(k \| m)$ when the key length is known or guessed.

Why the Shape of the Construction Matters

For the basic construction that outputs the full chaining state:

$$H(m) = h_t \quad \Longrightarrow \quad \text{attacker can continue the iteration.}$$

$$h_{t+1} = C(h_t, M_{t+1}), \quad h_{t+2} = C(h_{t+1}, M_{t+2}), \ldots$$

- Given $H(m)$ and $|m|$, compute $H(m \| \mathrm{pad}(m) \| x)$ without knowing m.
- Here $\mathrm{pad}(m)$ is the original padding: the attacker extends the already-padded message, not simply m.
- This is **length extension**; it does not produce a collision.
- It can break naive secret-prefix authentication $H(k \| m)$ when the key length is known or guessed.

Why the Shape of the Construction Matters

For the basic construction that outputs the full chaining state:

$$H(m) = h_t \quad \Longrightarrow \quad \text{attacker can continue the iteration.}$$

$$h_{t+1} = C(h_t, M_{t+1}), \quad h_{t+2} = C(h_{t+1}, M_{t+2}), \ldots$$

- Given $H(m)$ and $|m|$, compute $H(m \| \mathrm{pad}(m) \| x)$ without knowing m.
- Here $\mathrm{pad}(m)$ is the original padding: the attacker extends the already-padded message, not simply m.
- This is **length extension**; it does not produce a collision.
- It can break naive secret-prefix authentication $H(k \| m)$ when the key length is known or guessed.

Design question

Can we process long messages without exposing the entire state needed to continue hashing?

Outline

1. Constructing practical block ciphers
2. Substitution–permutation networks and AES
3. A bridge to practical hash functions
4. Sponge functions
5. Keccak, SHA-3, and SHAKE

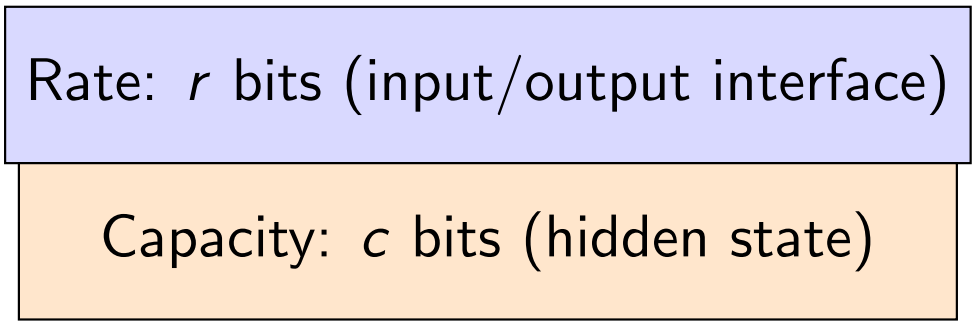
Why a "Sponge"?



Absorb the input, then **squeeze** out the output.

A Sponge: One State, Two Interfaces

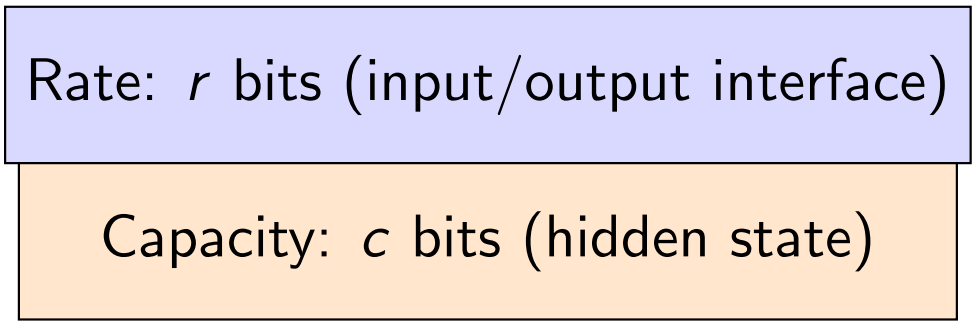
Use a public permutation $f : \{0,1\}^b \to \{0,1\}^b$.



$b = r + c$

A Sponge: One State, Two Interfaces

Use a public permutation $f : \{0,1\}^b \to \{0,1\}^b$.

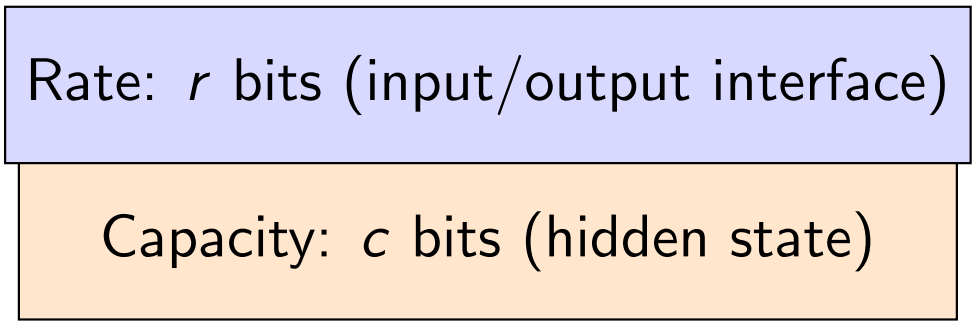


$b = r + c$

- Initialize all b state bits to zero.

A Sponge: One State, Two Interfaces

Use a public permutation $f : \{0,1\}^b \to \{0,1\}^b$.

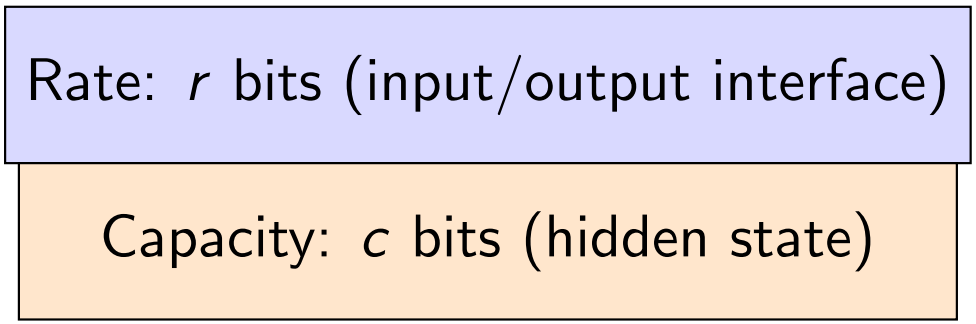


$$b = r + c$$

- Initialize all b state bits to zero.
- **Absorb:** XOR message blocks into the rate, then apply f.

A Sponge: One State, Two Interfaces

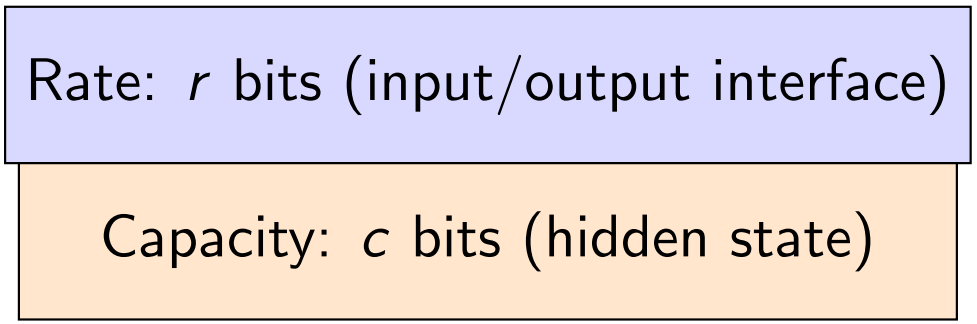
Use a public permutation $f : \{0,1\}^b \to \{0,1\}^b$.



- Initialize all b state bits to zero.
- **Absorb:** XOR message blocks into the rate, then apply f.
- **Squeeze:** read the rate, applying f between output blocks.

A Sponge: One State, Two Interfaces

Use a public permutation $f : \{0,1\}^b \to \{0,1\}^b$.

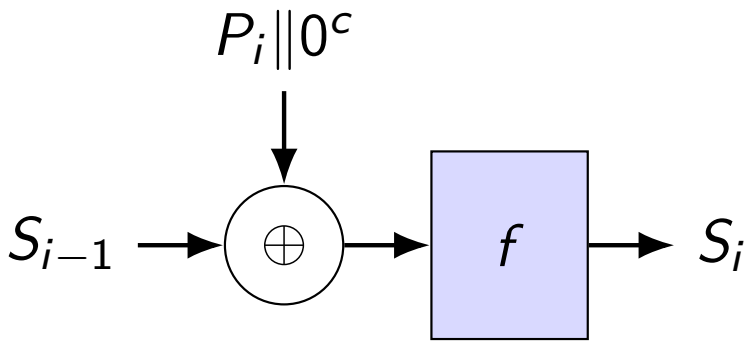


- Initialize all b state bits to zero.
- **Absorb:** XOR message blocks into the rate, then apply f.
- **Squeeze:** read the rate, applying f between output blocks.
- Every application of f mixes the *entire* state.

Absorbing the Message

Assume standard preprocessing has unambiguously encoded and padded the input into r-bit blocks $P_1, \ldots, P_t$.

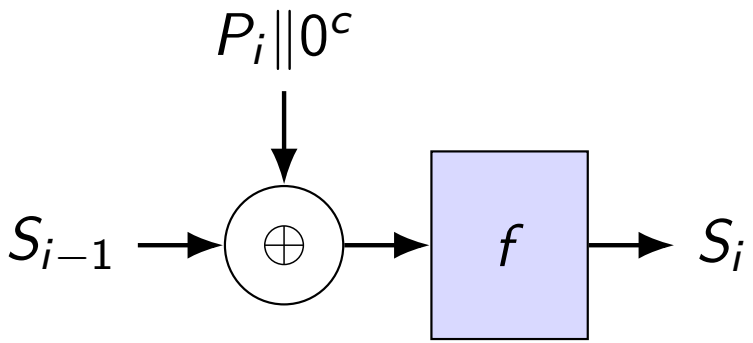
$$S_0 = 0^b, \qquad S_i = f\left(S_{i-1} \oplus (P_i \| 0^c)\right).$$



Absorbing the Message

Assume standard preprocessing has unambiguously encoded and padded the input into r-bit blocks $P_1, \ldots, P_t$.

$$S_0 = 0^b, \qquad S_i = f\left(S_{i-1} \oplus (P_i \| 0^c)\right).$$

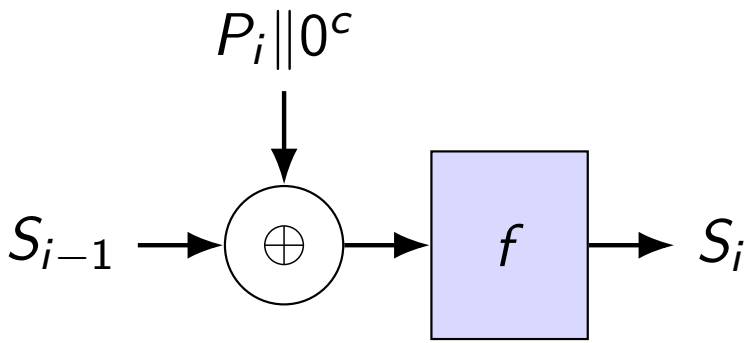


- The message is XORed into the rate only.

Absorbing the Message

Assume standard preprocessing has unambiguously encoded and padded the input into r-bit blocks $P_1, \ldots, P_t$.

$$S_0 = 0^b, \qquad S_i = f\left(S_{i-1} \oplus (P_i \| 0^c)\right).$$

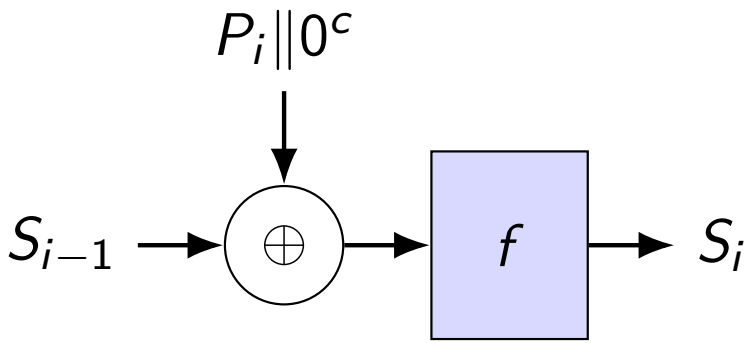


- The message is XORed into the rate only.
- The capacity also changes when f is applied.

Absorbing the Message

Assume standard preprocessing has unambiguously encoded and padded the input into r-bit blocks $P_1, \ldots, P_t$.

$$S_0 = 0^b, \qquad S_i = f\left(S_{i-1} \oplus (P_i \| 0^c)\right).$$

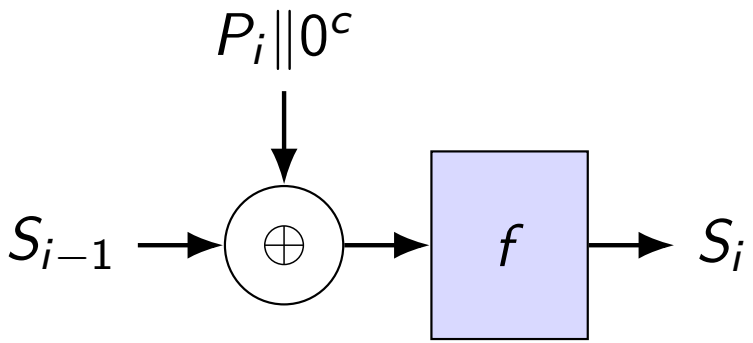


- The message is XORed into the rate only.
- The capacity also changes when f is applied.
- Standard preprocessing also distinguishes the different hash functions; we omit its bit-level details.

Absorbing the Message

Assume standard preprocessing has unambiguously encoded and padded the input into r-bit blocks $P_1, \ldots, P_t$.

$$S_0 = 0^b, \qquad S_i = f\left(S_{i-1} \oplus (P_i \| 0^c)\right).$$



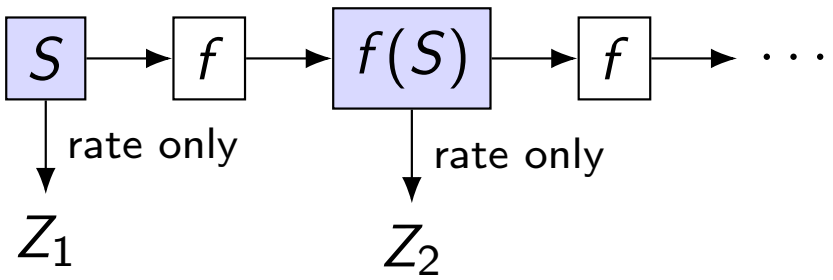
- The message is XORed into the rate only.
- The capacity also changes when f is applied.
- Standard preprocessing also distinguishes the different hash functions; we omit its bit-level details.
- Process the last input block before producing any output.

Squeezing the Output

Let $S = S_t$ after absorbing the last padded block.

$$Z_1 = \mathrm{rate}(S), \quad Z_2 = \mathrm{rate}(f(S)), \quad Z_3 = \mathrm{rate}(f^2(S)), \ldots$$

Return the first d bits of $Z_1 \| Z_2 \| Z_3 \| \cdots$.



No extra call to f before Z_1; one call between output blocks.

Exercise: Trace a Small Sponge

Toy parameters: $b = 16$, $r = 8$, $c = 8$; f is a public permutation.

Already-padded input and requested output

$P_1 = \texttt{A6}$, $P_2 = \texttt{3C}$; request $d = 24$ output bits.
Each hexadecimal pair represents 8 bits; the rate is written first.

Exercise: Trace a Small Sponge

Toy parameters: $b = 16$, $r = 8$, $c = 8$; f is a public permutation.

Already-padded input and requested output

$P_1 = \texttt{A6}$, $P_2 = \texttt{3C}$; request $d = 24$ output bits.
Each hexadecimal pair represents 8 bits; the rate is written first.

1. What are the two absorbing-state updates starting from 0000.

Exercise: Trace a Small Sponge

Toy parameters: $b = 16$, $r = 8$, $c = 8$; f is a public permutation.

Already-padded input and requested output

$P_1 = \texttt{A6}$, $P_2 = \texttt{3C}$; request $d = 24$ output bits.
Each hexadecimal pair represents 8 bits; the rate is written first.

1. What are the two absorbing-state updates starting from 0000.
2. Which bits are emitted? How many output blocks are needed?

Exercise: Trace a Small Sponge

Toy parameters: $b = 16$, $r = 8$, $c = 8$; f is a public permutation.

Already-padded input and requested output

$P_1 = \texttt{A6}$, $P_2 = \texttt{3C}$; request $d = 24$ output bits.
Each hexadecimal pair represents 8 bits; the rate is written first.

1. What are the two absorbing-state updates starting from 0000.
2. Which bits are emitted? How many output blocks are needed?
3. How many calls to f are needed in total?

Tracing the Sponge: Discussion

$$S_1 = f(\texttt{A600}), \qquad S_2 = f\big(S_1 \oplus \texttt{3C00}\big).$$

$$Z_1 = \mathrm{rate}(S_2), \quad Z_2 = \mathrm{rate}(f(S_2)), \quad Z_3 = \mathrm{rate}(f^2(S_2)).$$

- Output: $Z_1 \| Z_2 \| Z_3$ (three complete 8-bit blocks).
- Calls: 2 for absorption, then 2 more for squeezing.
- The tiny capacity is for tracing only; this toy has no useful security.

Why Keep a Capacity?

Why Keep a Capacity?

- If the full state were exposed, an attacker could evaluate f or f^{-1} on it and continue processing.

Why Keep a Capacity?

- If the full state were exposed, an attacker could evaluate f or f^{-1} on it and continue processing.
- Exposing only the rate does not directly reveal the capacity; invertibility of f alone does not recover the state.

Why Keep a Capacity?

- If the full state were exposed, an attacker could evaluate f or f^{-1} on it and continue processing.
- Exposing only the rate does not directly reveal the capacity; invertibility of f alone does not recover the state.
- At fixed b, a larger rate processes more input per call, but leaves a smaller capacity.

Why Keep a Capacity?

- If the full state were exposed, an attacker could evaluate f or f^{-1} on it and continue processing.
- Exposing only the rate does not directly reveal the capacity; invertibility of f alone does not recover the state.
- At fixed b, a larger rate processes more input per call, but leaves a smaller capacity.

Generic security intuition

Increasng capacity stengthens the security guarantee, but reduces throughput.

Why Keep a Capacity?

- If the full state were exposed, an attacker could evaluate f or f^{-1} on it and continue processing.
- Exposing only the rate does not directly reveal the capacity; invertibility of f alone does not recover the state.
- At fixed b, a larger rate processes more input per call, but leaves a smaller capacity.

Generic security intuition

Increasng capacity stengthens the security guarantee, but reduces throughput.

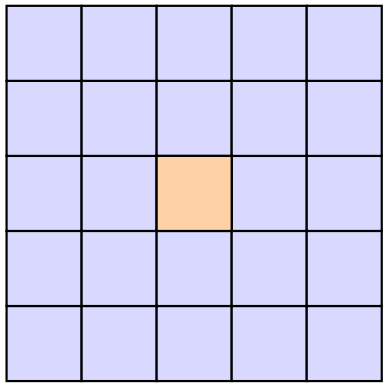
A concrete f needs cryptanalysis; hiding state alone is insufficient.

Outline

1. Constructing practical block ciphers
2. Substitution–permutation networks and AES
3. A bridge to practical hash functions
4. Sponge functions
5. Keccak, SHA-3, and SHAKE

Keccak Instantiates the Public Permutation

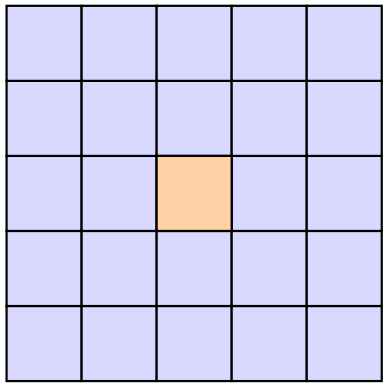
- SHA-3 and SHAKE use Keccak-f[1600]: a 1600-bit permutation with 24 rounds.
- State: 5×5 lanes of 64 bits each.
- Rounds use XORs, rotations, nonlinear operations, and constants.
- No secret key or key schedule.



Each square: one 64-bit lane

Keccak Instantiates the Public Permutation

- SHA-3 and SHAKE use Keccak-f[1600]: a 1600-bit permutation with 24 rounds.
- State: 5×5 lanes of 64 bits each.
- Rounds use XORs, rotations, nonlinear operations, and constants.
- No secret key or key schedule.



Each square: one 64-bit lane

Compare with AES

Both repeat nonlinearity and diffusion. AES uses a secret key to select a permutation; a sponge uses a public permutation with a restricted state interface.

Inside a Keccak Round: Roles, Not Bit Arithmetic

$$\theta \longrightarrow \rho \longrightarrow \pi \longrightarrow \chi \longrightarrow \iota$$

- θ XOR column-parity information into the state: diffusion.
- ρ Rotate each lane by a prescribed offset.
- π Move lanes to different positions: spread interactions.
- χ Nonlinear mixing within rows, using NOT, AND, and XOR.
- ι XOR a round constant into one lane: break symmetries.

The design pattern should look familiar

Local nonlinearity + structured mixing + repeated rounds. Unlike AES, the asymmetry here comes from public round constants rather than secret round keys.

One Permutation, Several Standardized Functions

Function	Rate r	Capacity c	Output bits
SHA3-224	1152	448	224
SHA3-256	1088	512	256
SHA3-384	832	768	384
SHA3-512	576	1024	512
SHAKE128	1344	256	chosen d
SHAKE256	1088	512	chosen d

- All use the same 1600-bit permutation; $r + c = 1600$.
- SHA-3 fixes the digest length; SHAKE is an extendable-output function (XOF).
- Function-specific preprocessing matters: SHA3-256 and the original Keccak-256 are different functions.

Output Length Is Not the Whole Security Story

- An ideal d-bit digest permits birthday collisions in about $2^{d/2}$ work.

Output Length Is Not the Whole Security Story

- An ideal d-bit digest permits birthday collisions in about $2^{d/2}$ work.
- Capacity constrains the sponge's generic security guarantee too.

Output Length Is Not the Whole Security Story

- An ideal d-bit digest permits birthday collisions in about $2^{d/2}$ work.
- Capacity constrains the sponge's generic security guarantee too.
- SHA3-256 targets 128-bit collision and 256-bit preimage security.

Output Length Is Not the Whole Security Story

- An ideal d-bit digest permits birthday collisions in about $2^{d/2}$ work.
- Capacity constrains the sponge's generic security guarantee too.
- SHA3-256 targets 128-bit collision and 256-bit preimage security.
- SHAKE128 with d output bits targets $\min(d/2, 128)$ bits of collision security.

Output Length Is Not the Whole Security Story

- An ideal d-bit digest permits birthday collisions in about $2^{d/2}$ work.
- Capacity constrains the sponge's generic security guarantee too.
- SHA3-256 targets 128-bit collision and 256-bit preimage security.
- SHAKE128 with d output bits targets $\min(d/2, 128)$ bits of collision security.

Quick check

How much collision security does SHAKE128 provide with $d = 128$? With $d = 256$? With $d = 512$?

Output Length Is Not the Whole Security Story

- An ideal d-bit digest permits birthday collisions in about $2^{d/2}$ work.
- Capacity constrains the sponge's generic security guarantee too.
- SHA3-256 targets 128-bit collision and 256-bit preimage security.
- SHAKE128 with d output bits targets $\min(d/2, 128)$ bits of collision security.

Quick check

How much collision security does SHAKE128 provide with $d = 128$? With $d = 256$? With $d = 512$?

64, 128, and 128 bits, respectively. More output does not increase the target indefinitely.

XOFs Produce a Stream, Not Independent Digests

For the same input m:

$$\mathrm{SHAKE128}(m, 256) = \text{first 256 bits of } \mathrm{SHAKE128}(m, 512).$$

XOFs Produce a Stream, Not Independent Digests

For the same input m:

$$\mathrm{SHAKE128}(m, 256) = \text{first 256 bits of } \mathrm{SHAKE128}(m, 512).$$

- Choosing a longer output continues squeezing the same stream.

XOFs Produce a Stream, Not Independent Digests

For the same input m:

$$\mathrm{SHAKE128}(m, 256) = \text{first 256 bits of } \mathrm{SHAKE128}(m, 512).$$

- Choosing a longer output continues squeezing the same stream.
- Useful when an application needs a specified amount of derived output.

XOFs Produce a Stream, Not Independent Digests

For the same input m:

$$\mathrm{SHAKE128}(m, 256) = \text{first 256 bits of } \mathrm{SHAKE128}(m, 512).$$

- Choosing a longer output continues squeezing the same stream.
- Useful when an application needs a specified amount of derived output.
- Different purposes need domain separation, rather than assuming different output lengths give independent values.

XOFs Produce a Stream, Not Independent Digests

For the same input m:

$$\mathrm{SHAKE128}(m, 256) = \text{first 256 bits of } \mathrm{SHAKE128}(m, 512).$$

- Choosing a longer output continues squeezing the same stream.
- Useful when an application needs a specified amount of derived output.
- Different purposes need domain separation, rather than assuming different output lengths give independent values.
- Standardized related constructions include cSHAKE (customization) and KMAC (keyed authentication).

XOFs Produce a Stream, Not Independent Digests

For the same input m:

$$\mathrm{SHAKE128}(m, 256) = \text{first 256 bits of } \mathrm{SHAKE128}(m, 512).$$

- Choosing a longer output continues squeezing the same stream.
- Useful when an application needs a specified amount of derived output.
- Different purposes need domain separation, rather than assuming different output lengths give independent values.
- Standardized related constructions include cSHAKE (customization) and KMAC (keyed authentication).

No free secret randomness

Public input to a public deterministic XOF produces public output. Long output does not create entropy or secrecy.

Returning to Length Extension

- In basic Merkle–Damgård, the digest exposes the full chaining state.

Returning to Length Extension

- In basic Merkle–Damgård, the digest exposes the full chaining state.
- In SHA-3, the digest does not expose the full sponge state; capacity remains hidden.

Returning to Length Extension

- In basic Merkle–Damgård, the digest exposes the full chaining state.
- In SHA-3, the digest does not expose the full sponge state; capacity remains hidden.
- SHA-3 is designed to resist the usual digest-based length-extension attack.

Returning to Length Extension

- In basic Merkle–Damgård, the digest exposes the full chaining state.
- In SHA-3, the digest does not expose the full sponge state; capacity remains hidden.
- SHA-3 is designed to resist the usual digest-based length-extension attack.
- SHAKE's prefix relation is about output for the *same input*, not extending the input message.

Returning to Length Extension

- In basic Merkle–Damgård, the digest exposes the full chaining state.
- In SHA-3, the digest does not expose the full sponge state; capacity remains hidden.
- SHA-3 is designed to resist the usual digest-based length-extension attack.
- SHAKE's prefix relation is about output for the *same input*, not extending the input message.

Keep the security goals separate

Resistance to length extension is not a proof that an arbitrary keyed hash expression is a secure MAC. Use a construction with its own authentication analysis, such as HMAC or KMAC.